ECS Guardian: A Serverless Solution for Container Resource Lifecycle Management 🛡️

Automate detection of unused images, stale task definition revisions, and idle Fargate services with human-approved remediation

📘 Introduction

Container platforms accumulate waste quietly. Every deployment pushes a new image to Amazon ECR and registers a new ECS task definition revision. Over months, old images pile up and incur storage charges, hundreds of stale revisions clutter the console, and services that nobody uses keep running on Fargate around the clock.

ECS Guardian is a serverless housekeeping tool that finds this waste and reports on it. A single Lambda function scans ECR and ECS, works out what is still in use, and emails a cost-savings report through SNS. It runs in two modes:

- **Report mode (dry run):** lists everything it would remove and deletes nothing.
- **Cleanup mode:** deletes unused images and deregisters stale task definition revisions, but only when a human triggers it.

Idle ECS services are only ever reported, never touched automatically. The Guardian is conservative by design: it never deletes an image that a running task or a retained task definition still references.

📋 Prerequisites

Account and access

- An AWS account with permissions to create IAM roles and policies, and to use ECR, ECS, Lambda, SNS, EventBridge Scheduler, and CloudWatch
- Region set to **ap-south-1 (Mumbai)** or any region of your choice throughout
- A default VPC with public subnets, for the demo Fargate service

Tools

- AWS Management Console access
- AWS CloudShell, which provides the AWS CLI and Docker for building and pushing the demo images
- An email inbox you can access, to confirm the SNS subscription and receive reports

Knowledge

- Basic familiarity with containers, ECR repositories, and ECS task definitions and services
- Basic understanding of IAM roles and inline policies
- Ability to read Python (boto3) at a beginner level

Cost awareness

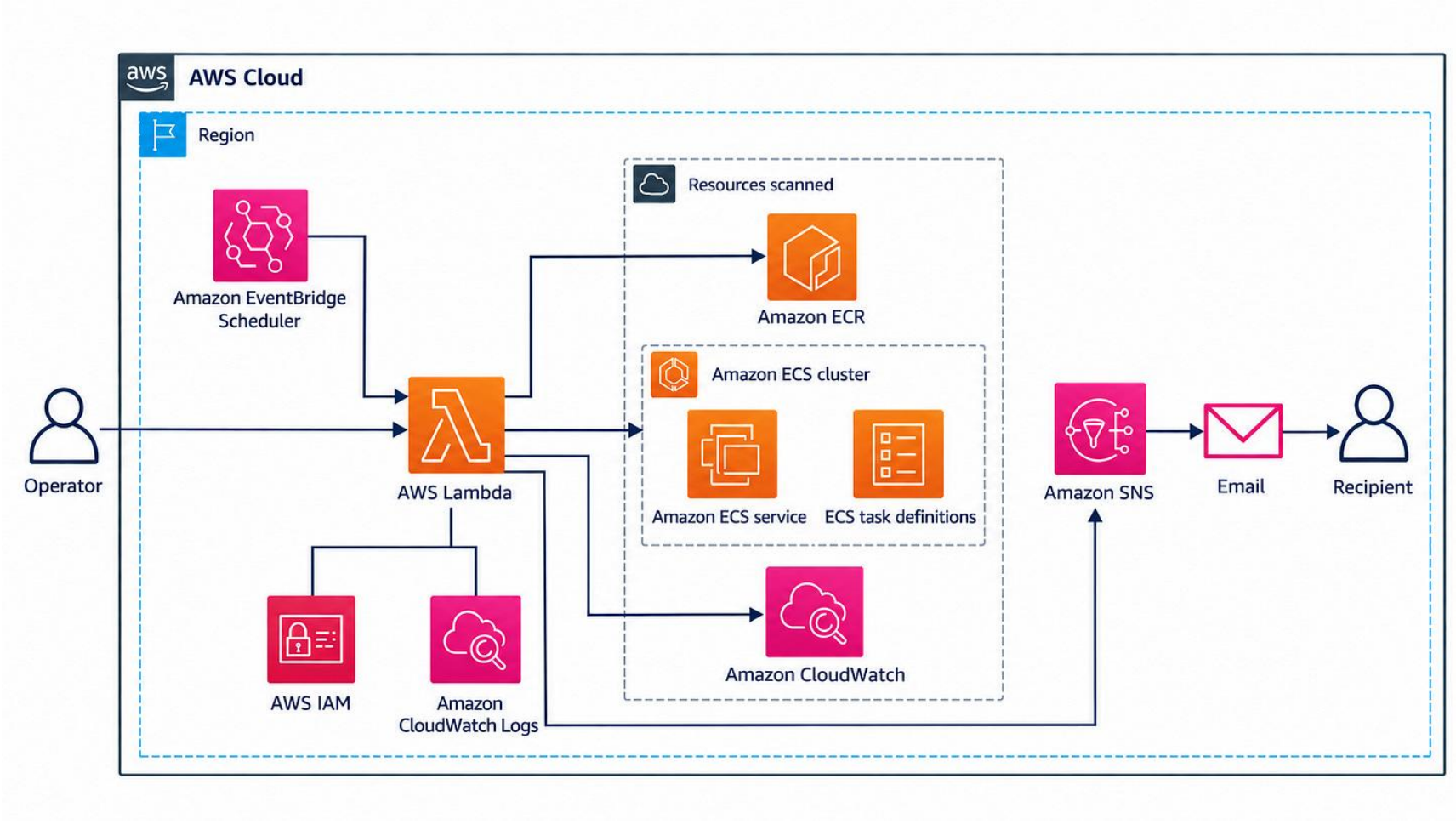
- The demo runs one small Fargate task (0.25 vCPU, 0.5 GB), so keep the lab running only as long as needed and complete the teardown in Step VIII.

🎯 Aim of the Project

The project aims to:

- Automatically detect **unused ECR images** that are older than a threshold and are not among the newest N images in their repository.
- Detect **stale ECS task definition revisions** beyond the newest N per family that are not used by any service or running task.
- Identify **idle ECS services**, meaning those whose maximum CPU stays below a threshold over a set number of days, and estimate their monthly cost.
- Protect anything in use by checking running tasks, service task definitions, and kept revisions before flagging an image.
- Estimate potential monthly savings and deliver the findings as an email report through SNS.
- Keep a human in the loop: nothing is deleted until cleanup mode is invoked, and a guardian:ignore=true tag exempts any repository or service.
- Run the report automatically every week through EventBridge Scheduler.

🏛️ Architecture Diagram

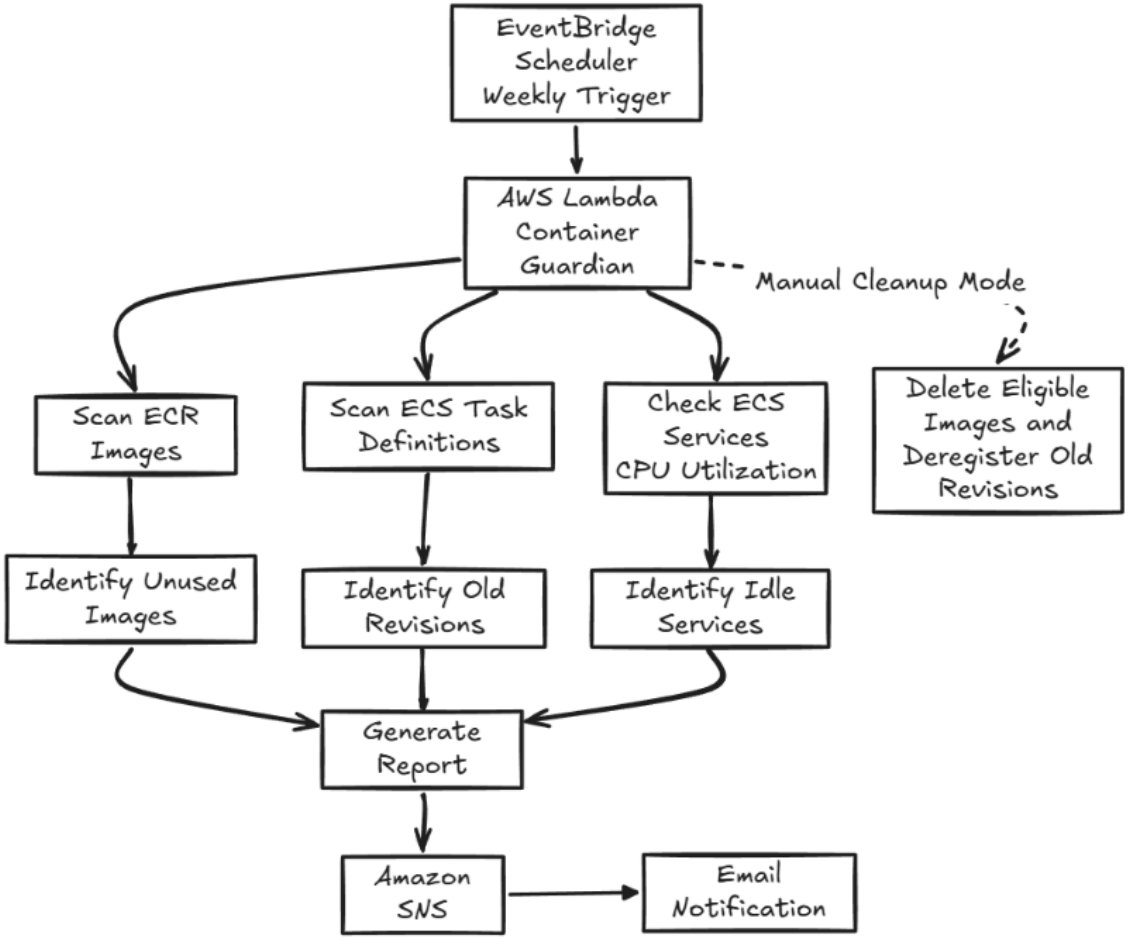


🧩 Architecture Components

- **Amazon EventBridge Scheduler (**ecs-guardian-weekly**)**: Triggers the Guardian every Monday at 9:00 AM IST (0 9 ? * MON *) with the payload {"mode": "report"}.
- **AWS Lambda (**container-guardian**)**: Python function (256 MB, 5 minute timeout) that scans ECR, ECS and CloudWatch, works out which resources are unused, optionally deletes them, and publishes the report.
- **Operation Modes**: report mode is a dry run that lists findings and deletes nothing. cleanup mode deletes unused images and deregisters stale task definitions, and runs only when triggered manually.
- **Lambda Environment Variables**: TOPIC_ARN, IMAGE_AGE_DAYS, KEEP_LATEST_IMAGES, KEEP_TASKDEF_REVISIONS, IDLE_CPU_PERCENT and IDLE_DAYS set the thresholds, so the same code runs with aggressive test values or conservative production values.
- **Amazon ECR (**guardian-demo-app**)**: Private repository holding four demo images (v1 to v4). The Guardian flags images that are old enough, outside the newest N, and not referenced by any running task or kept task definition.
- **Amazon ECS Cluster (**guardian-demo-cluster**)**: Fargate cluster that hosts the demo service, which the Guardian scans for running tasks and idle services.
- **Amazon ECS Service (**guardian-demo-svc**)**: Runs one Fargate task using the v2 image. It proves the Guardian never deletes an image that is in use.
- **ECS Task Definitions (**guardian-demo-task**)**: Six revisions, where revisions 1 to 4 use v1 to v4 and revisions 5 and 6 use v2. Revisions beyond the newest N that are not in use are marked stale and deregistered in cleanup mode.

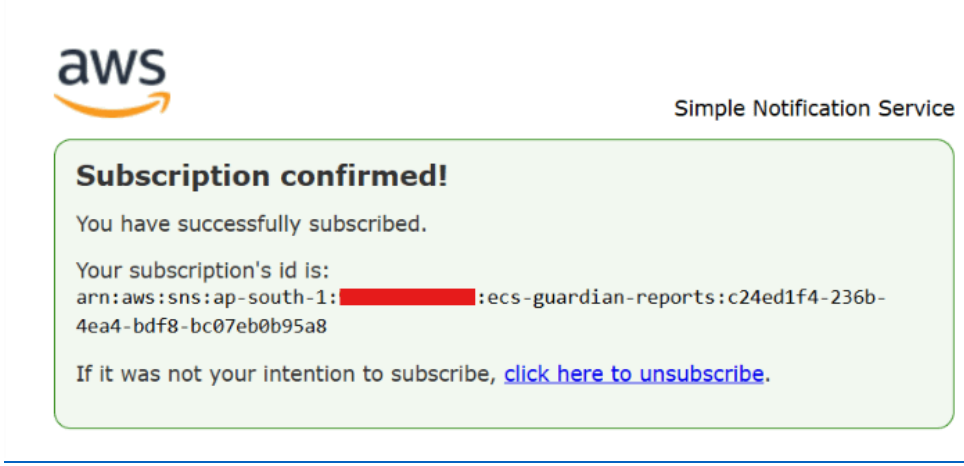
- **Amazon CloudWatch Metrics**: Provide AWS/ECS CPUUtilization data. A service whose maximum CPU stays below the threshold for IDLE_DAYS is reported as idle, with an estimated monthly cost.
- **Amazon SNS (**ecs-guardian-reports**) and Email Subscription**: Deliver the findings report (unused images, stale task definitions, idle services and estimated savings) to the subscribed email address.
- **Amazon CloudWatch Logs (**/aws/lambda/container-guardian**)**: Store the Lambda execution logs for troubleshooting.
- **IAM Role (**ContainerGuardianRole**)**: Grants the Lambda function least-privilege access, with read access to ECR, ECS and CloudWatch, delete and deregister actions for cleanup mode, and sns:Publish limited to the report topic.
- **IAM Role (**ecsTaskExecutionRole**)**: Lets the demo Fargate tasks pull images from ECR and write logs.
- **Resource Tag (**guardian:ignore=true**)**: Exempts any ECR repository or ECS service from scanning and cleanup.
- **AWS CloudShell**: Used once during setup to build and push the demo images and register the task definition revisions.

🛠️ Execution Workflow

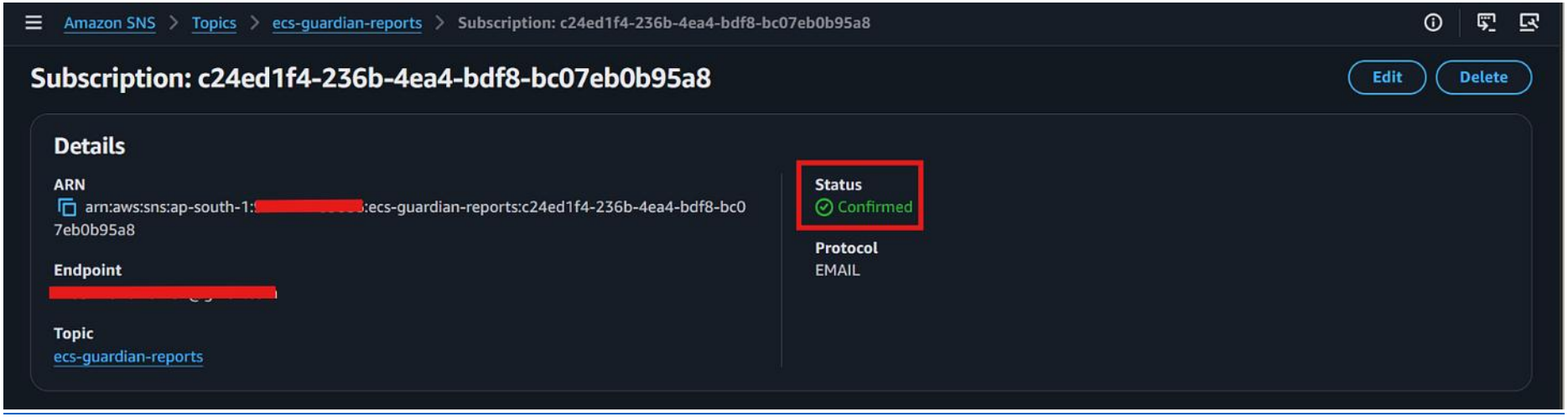


Step I: Configure SNS Notifications

- Navigate to Amazon SNS → Topics → Create topic.
- Configure the following:
  - Type: Standard
  - Name: ecs-guardian-reports
- Click Create topic.
- Copy the Topic ARN. You will need it in Step III and Step IV.
- Open the topic → Create subscription:
  - Protocol: Email
  - Endpoint: your email address
- Click Create subscription.
- Open the confirmation email from AWS and click Confirm subscription.



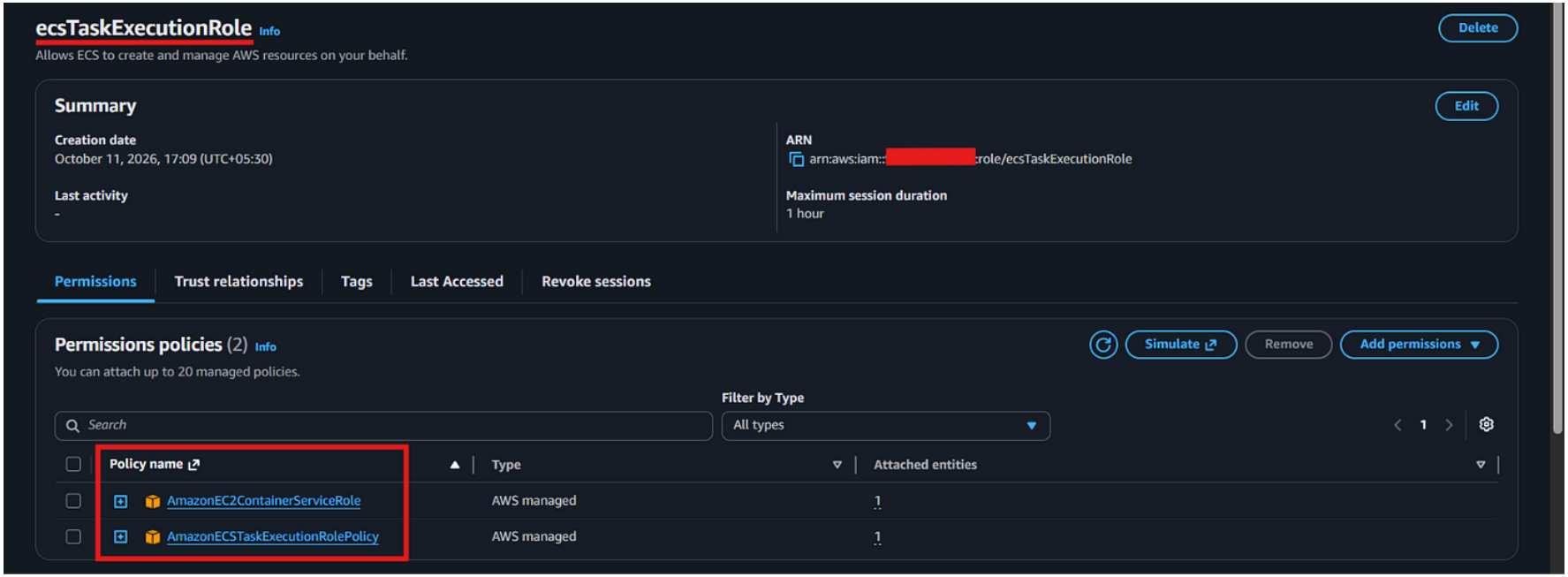
- Verify the subscription status shows Confirmed.



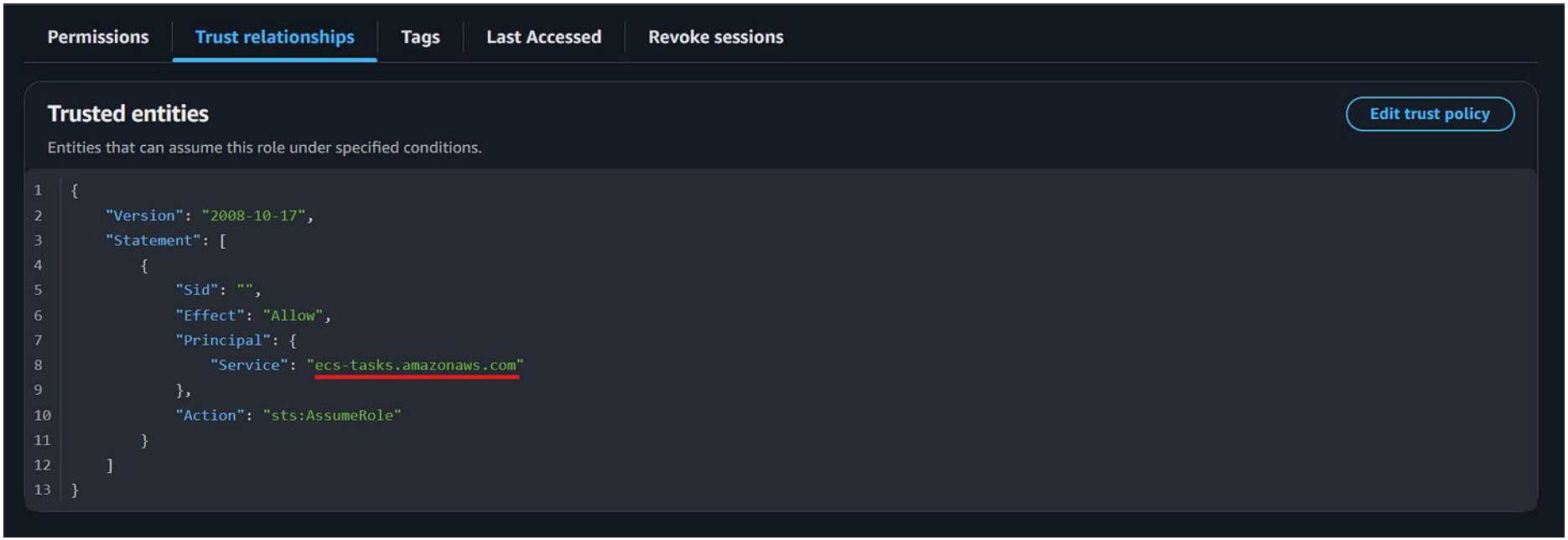
Step II: Provision Demo Resources

Create the Task Execution Role

- Navigate to IAM → Roles → Create role.
- Trusted entity type: AWS service
- Use case: Elastic Container Service → Elastic Container Service Task
- Attach policy: AmazonECSTaskExecutionRolePolicy
- Role name: ecsTaskExecutionRole
- Click Create role.

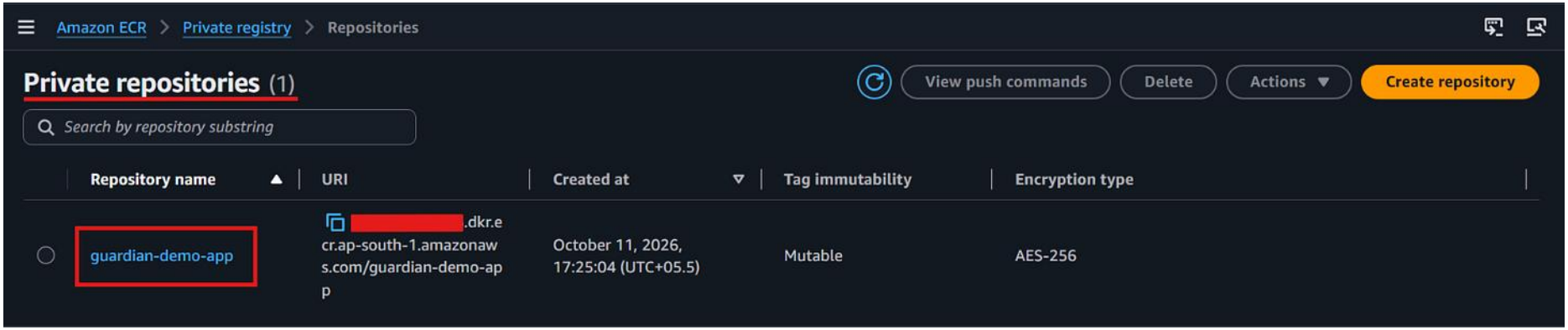


- Update the trust policy using the ***ecs-task-trust-policy.json*** as a reference.



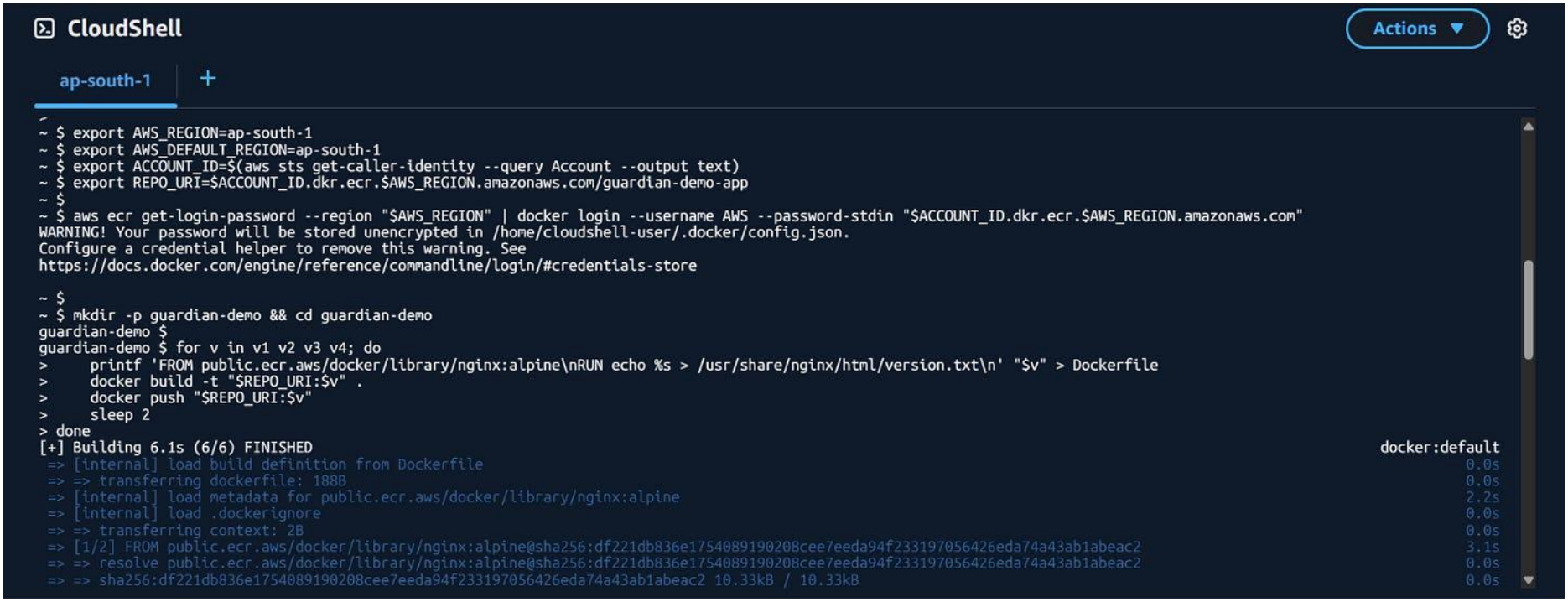
Create the ECR repository

- Navigate to Amazon ECR → Private registry → Repositories → Create repository.
- Repository name: guardian-demo-app
- Leave the other settings as default and click Create.

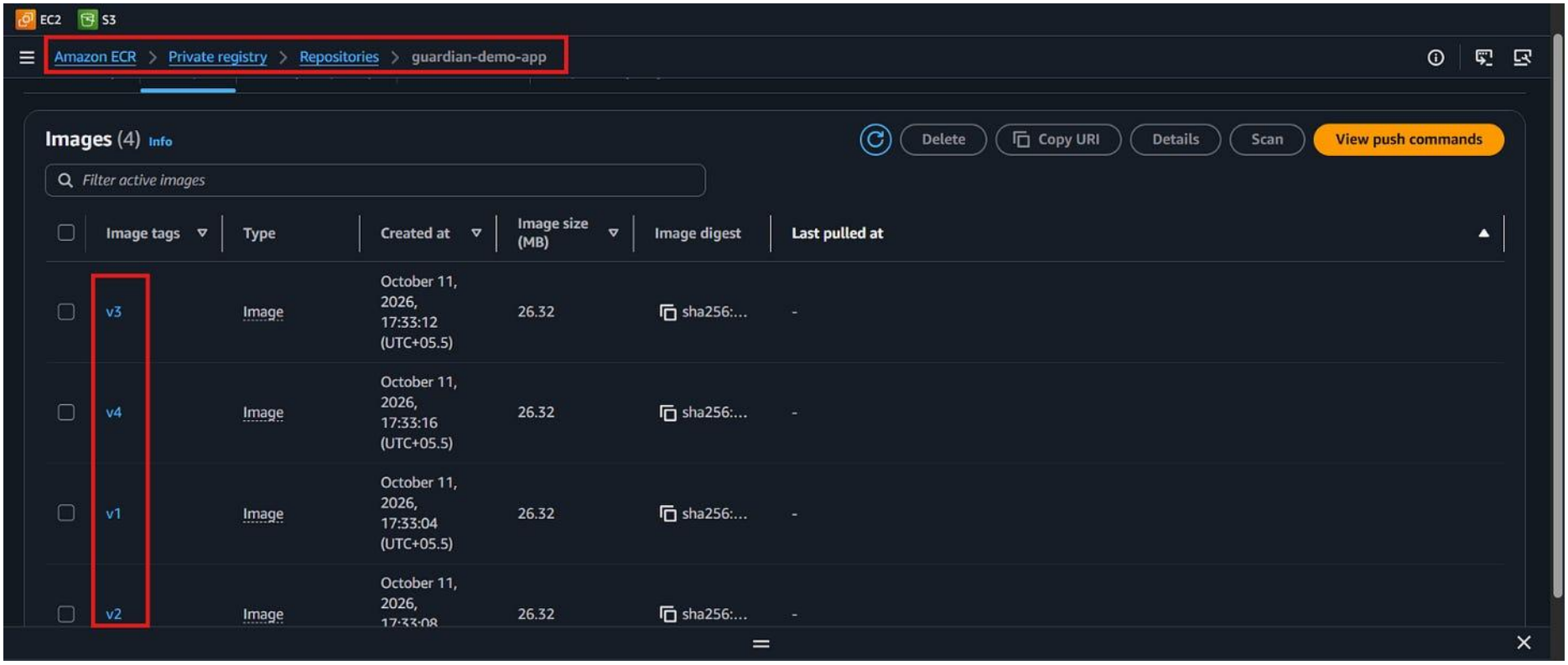


Push Container Images

- Open AWS CloudShell. Make sure the region is ap-south-1.
- Use the script provided in the GitHub repository below. It builds four versioned images (v1 to v4) and pushes them to your ECR repository: ***push-demo-images.sh***



- In ECR → guardian-demo-app, verify that 4 images exist with tags v1, v2, v3, v4.

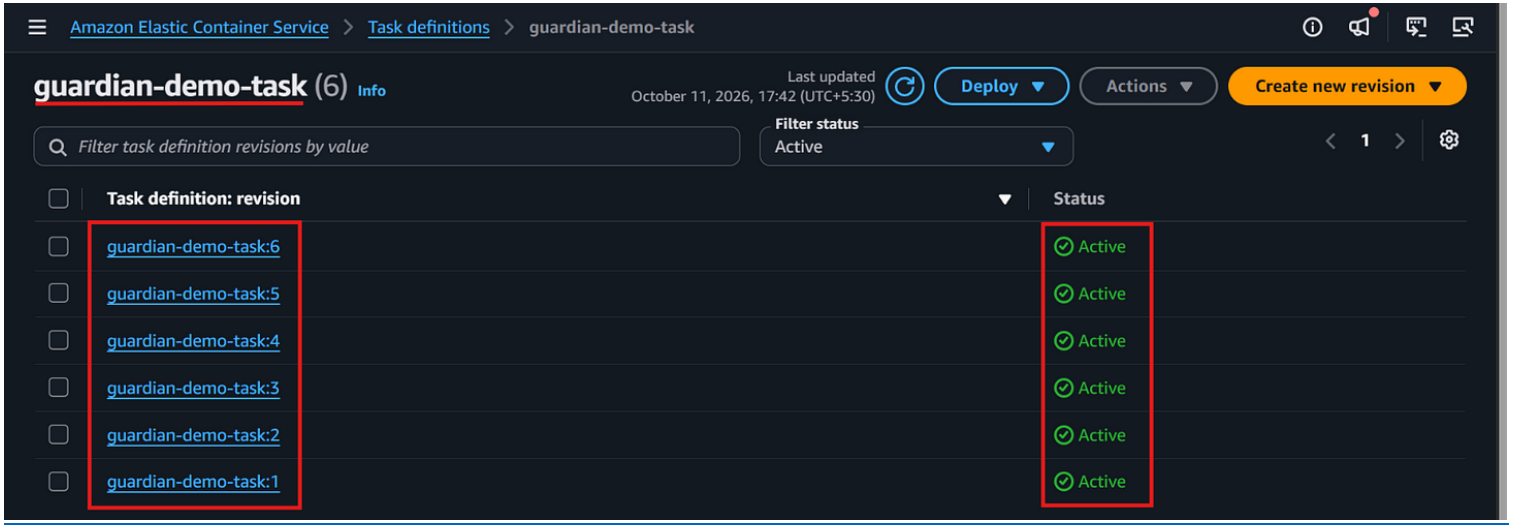


Register Task Definition Revisions

- In the same CloudShell session, run the script provided in the GitHub repository: ***register-task-definitions.sh***



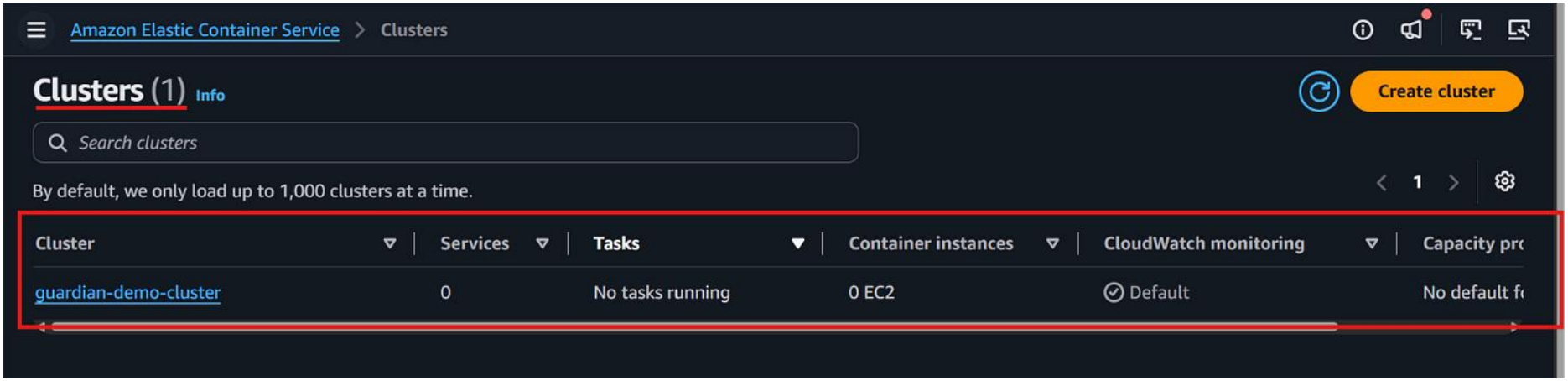
- Navigate to ECS → Task definitions → guardian-demo-task and verify revisions 1 to 6 exist.



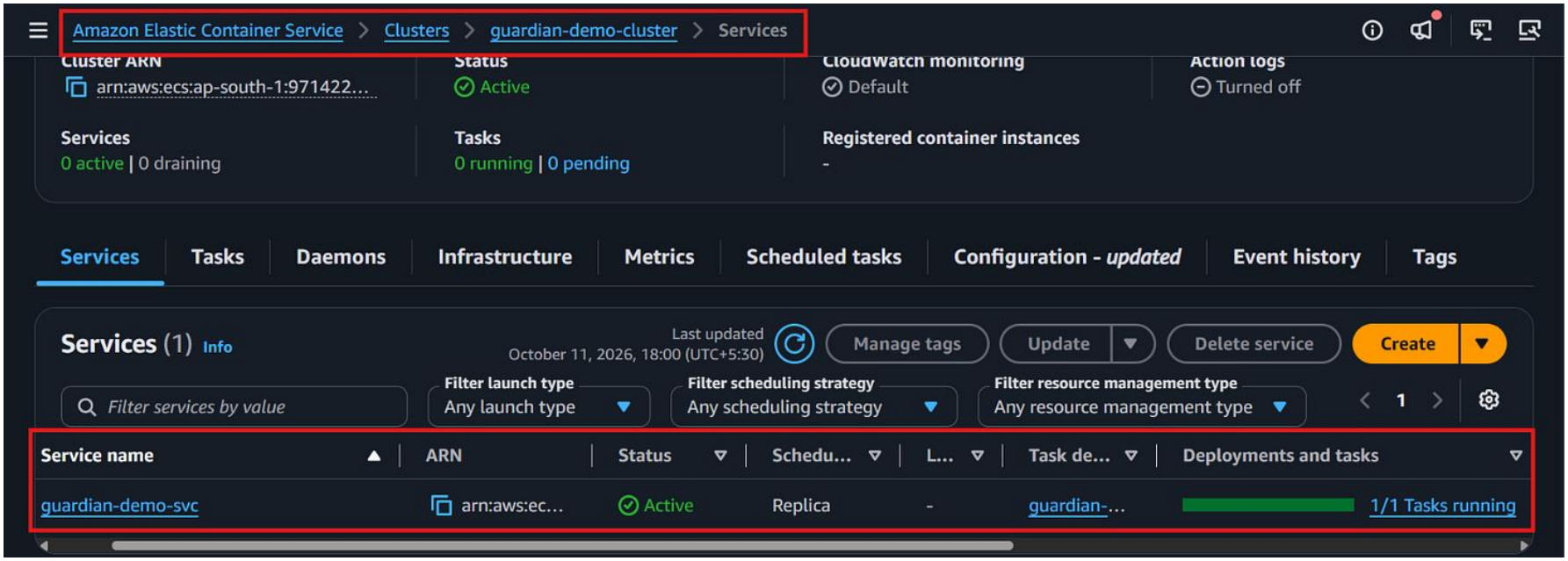
Revisions 1 to 4 use images v1 ***to*** v4 ***respectively, while revisions 5 and 6 both use*** v2

Create the ECS Cluster and Service

- Navigate to ECS → Clusters → Create cluster.
- Cluster name: guardian-demo-cluster
- Infrastructure: AWS Fargate (serverless)
- Click Create.



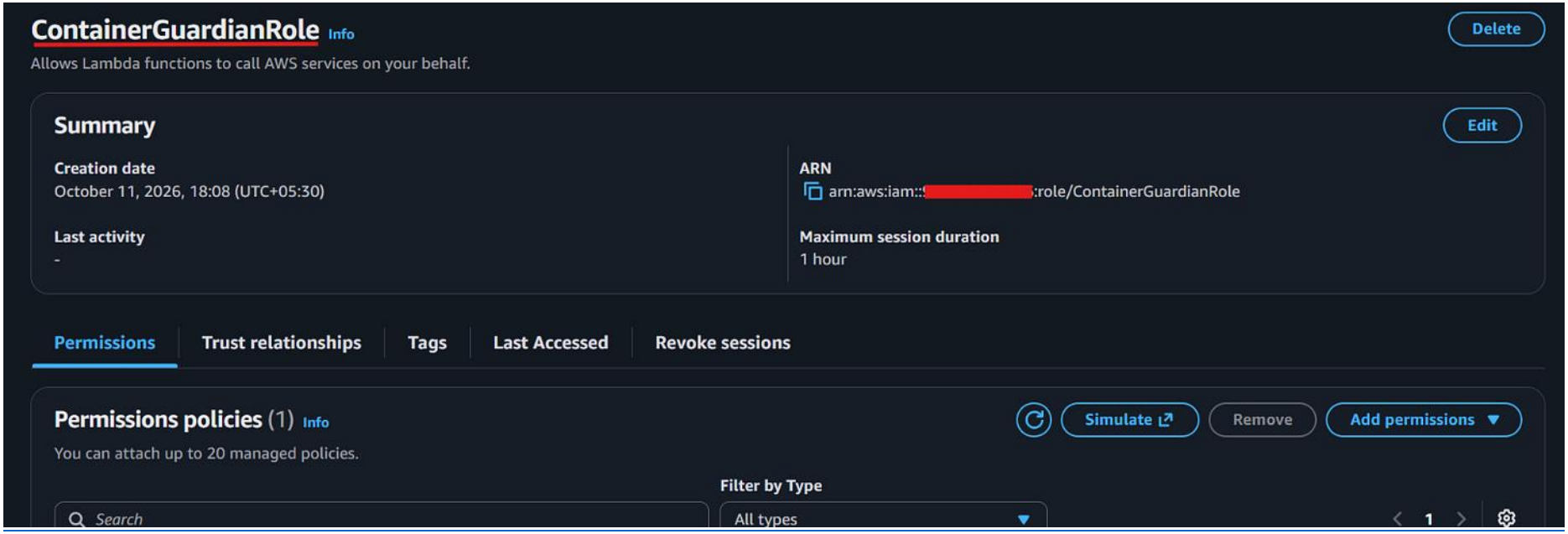
- Open the cluster → Services → Create.
- Configure the following:
  - Compute options: Launch type → FARGATE
  - Family: guardian-demo-task
  - Revision: 6
  - Service name: guardian-demo-svc
  - Desired tasks: 1
  - Networking: default VPC, public subnets, Public IP turned on
  - Security group: default is fine (no inbound traffic is needed)
- Click Create.
- Verify the service shows 1/1 tasks Running.



The service runs the v2 image, which is intentionally not the newest. This demonstrates that the Guardian never deletes an image that is still in use.

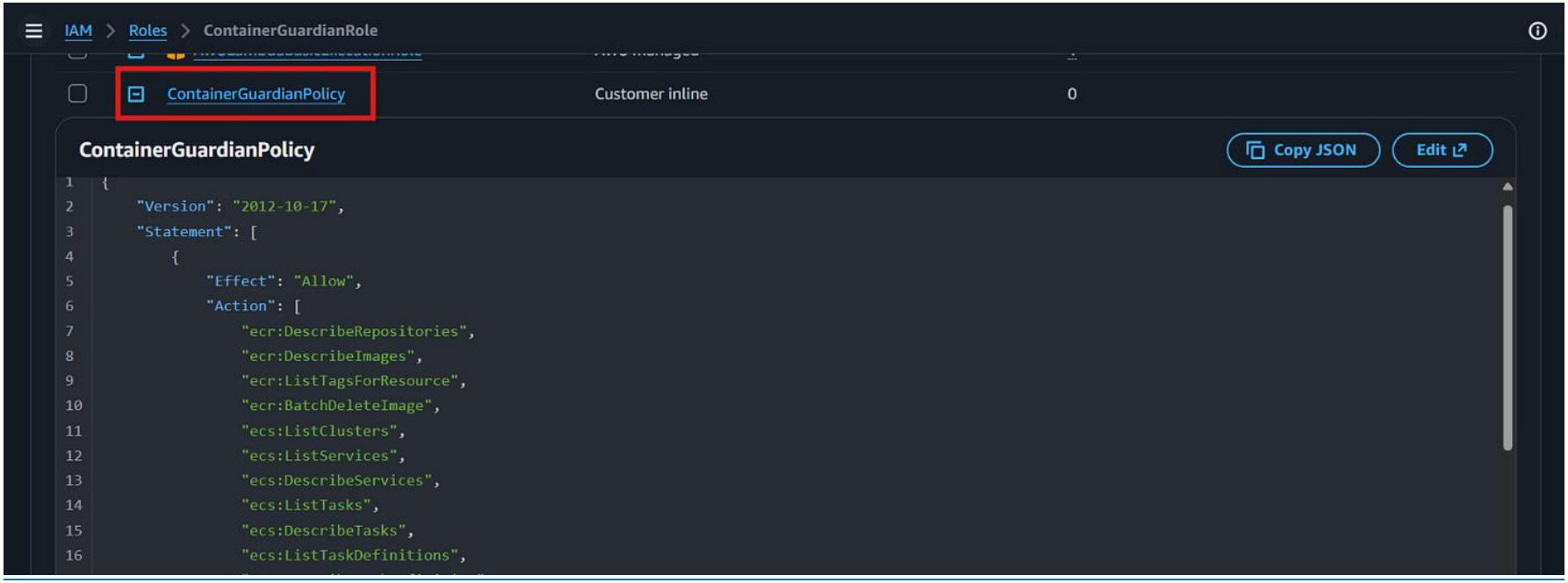
Step III: Create the Lambda IAM Role

- Navigate to IAM → Roles → Create role.
- Trusted entity type: AWS service
- Use case: Lambda
- Attach policy: AWSLambdaBasicExecutionRole
- Role name: ContainerGuardianRole
- Click Create role.



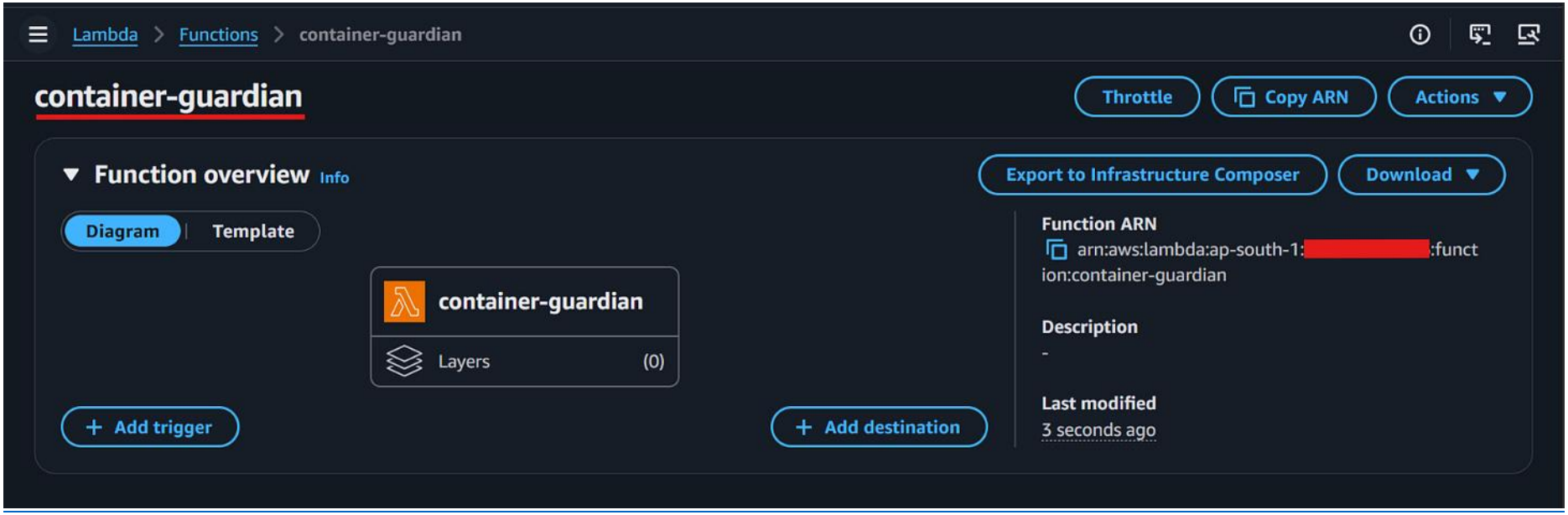
Attach the Inline Policy

- Open ContainerGuardianRole → Add permissions → Create inline policy.
- Switch to the JSON editor and paste the policy provided in the GitHub repository: ***lambda-execution-policy.json***
- Replace the SNS ARN with the Topic ARN from Step I.
- Policy name: ContainerGuardianPolicy
- Click Create policy.
- Verify ContainerGuardianPolicy appears under the role's Inline policies.

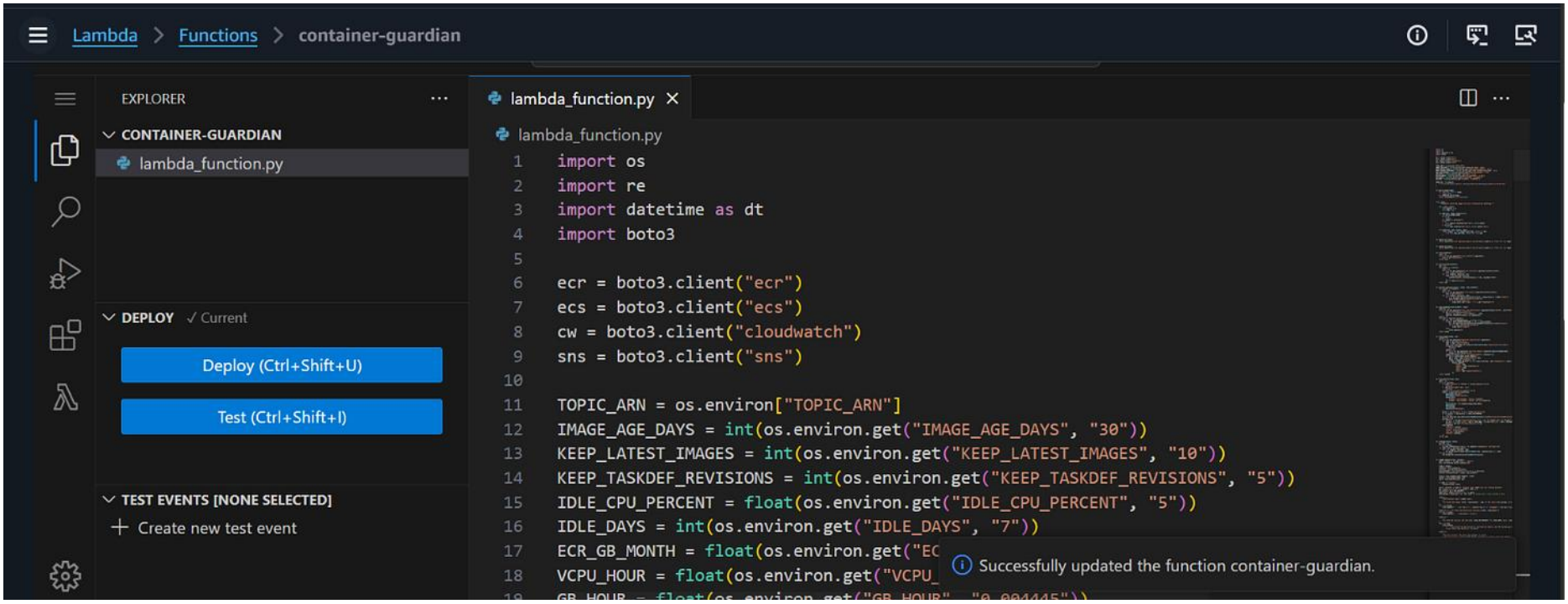


Step IV: Create the Lambda Function

- Navigate to AWS Lambda → Functions → Create function.
- Select Author from scratch and configure:
  - Function name: container-guardian
  - Runtime: Python 3.12
  - Execution role: Use an existing role → ContainerGuardianRole
- Click Create function.

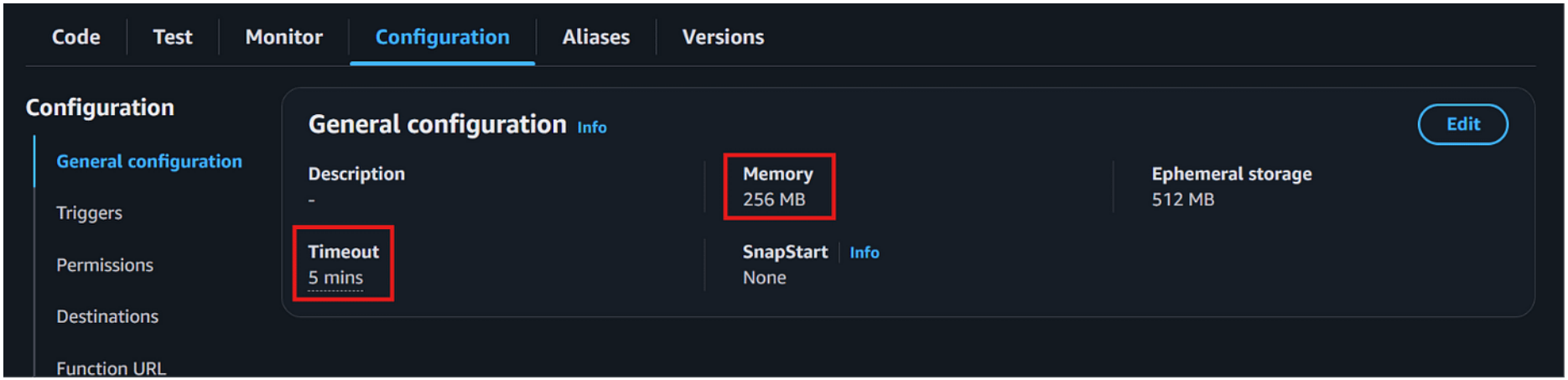


- In the Code tab, open lambda_function.py, delete the sample code, and paste the Guardian code from the GitHub repository below. It scans ECR, ECS and CloudWatch, then publishes the report to SNS: ***container_guardian.py***
- Click Deploy.



Configure the function

- Go to Configuration → General configuration → Edit:
  - Timeout: 5 min 0 sec
  - Memory: 256 MB



- Go to Configuration → Environment variables → Edit and add the following (these are **test values**, so the demo items get flagged right away):
  - TOPIC_ARN: Your SNS Topic ARN from Step I
  - IMAGE_AGE_DAYS: 0
  - KEEP_LATEST_IMAGES: 1
  - KEEP_TASKDEF_REVISIONS: 2
  - IDLE_CPU_PERCENT: 5

- IDLE_DAYS: 1

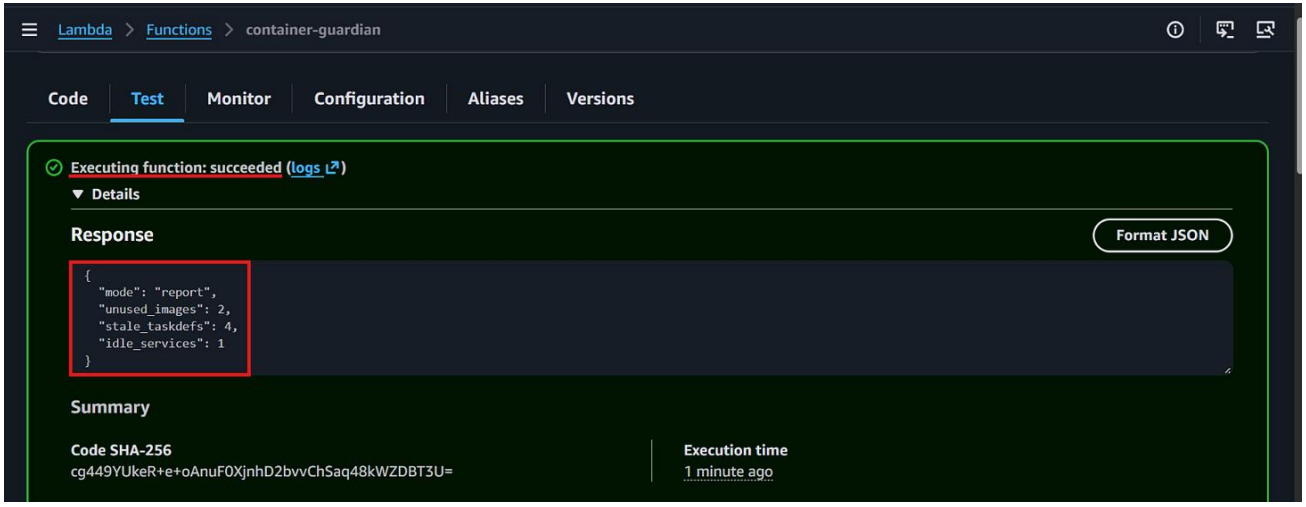
- Click Save.



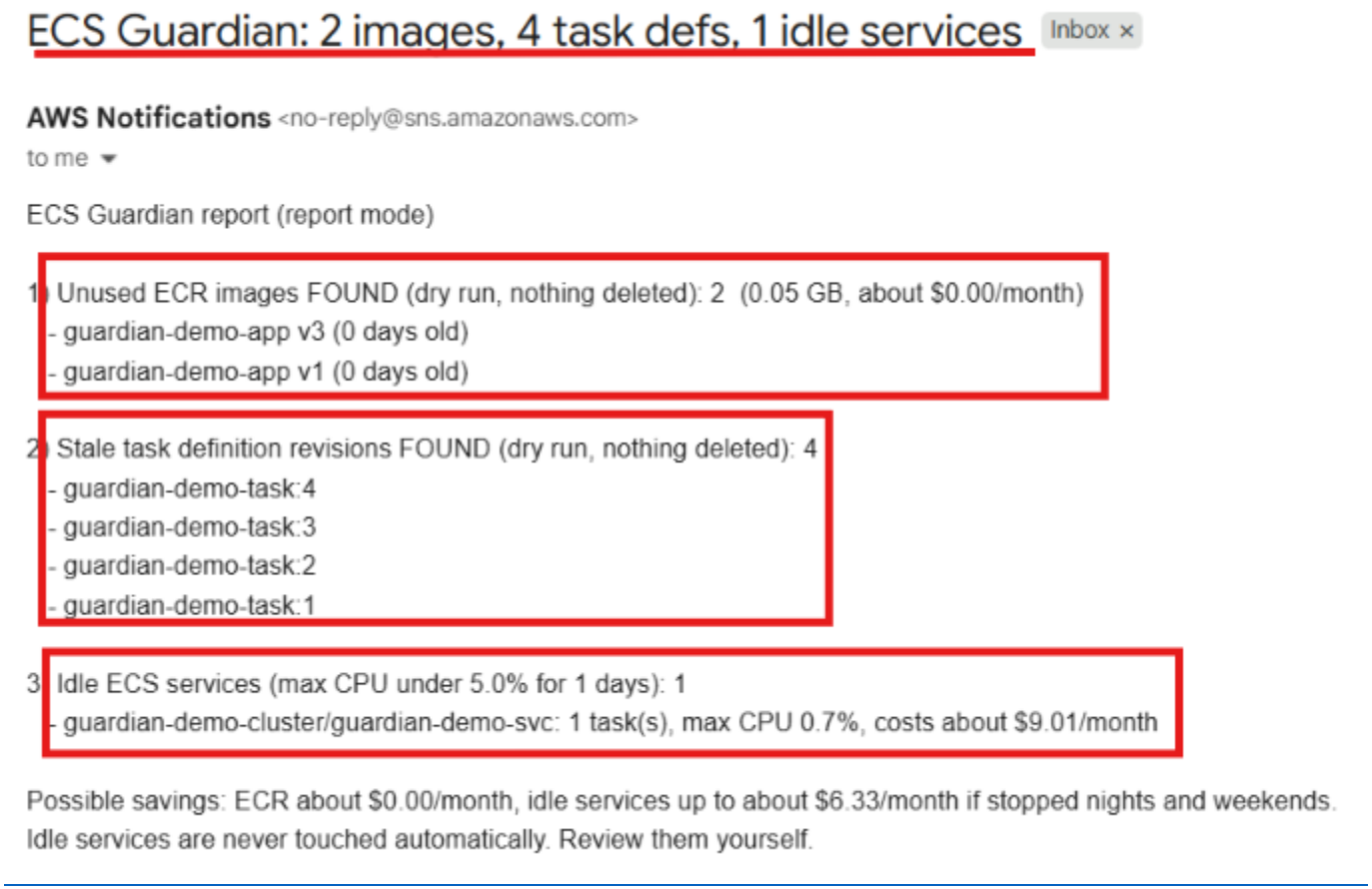
The Fargate price variables (VCPU_HOUR, GB_HOUR) default to US East rates. For accurate numbers, check the AWS Fargate pricing page for ap-south-1 and add those two variables.

Step V: Run Report Mode (Dry Run)

- Wait about 10 minutes after the ECS service started, so CloudWatch has CPU data for it.
- Open the Test tab in Lambda and create a test event:
  - Event name: report
  - Event JSON: {"mode": "report"}
- Click Test.
- Verify the result shows:
  - "unused_images": 2
  - "stale_taskdefs": 4
  - "idle_services": 1



- Open your email and read the report.



ECS Guardian: 2 images, 4 task defs, 1 idle services Inbox ×

AWS Notifications <no-reply@sns.amazonaws.com>
to me

ECS Guardian report (report mode)

1) Unused ECR images FOUND (dry run, nothing deleted): 2 (0.05 GB, about $0.00/month)
- guardian-demo-app v3 (0 days old)
- guardian-demo-app v1 (0 days old)

2) Stale task definition revisions FOUND (dry run, nothing deleted): 4
- guardian-demo-task:4
- guardian-demo-task:3
- guardian-demo-task:2
- guardian-demo-task:1

3) Idle ECS services (max CPU under 5.0% for 1 days): 1
- guardian-demo-cluster/guardian-demo-svc: 1 task(s), max CPU 0.7%, costs about $9.01/month

Possible savings: ECR about $0.00/month, idle services up to about $6.33/month if stopped nights and weekends.
Idle services are never touched automatically. Review them yourself.

Expected Results:

ECR images v1 ***and*** v3 ***flagged: Neither is running, and neither is referenced by a retained task definition.***

Image v2 ***not flagged: It is used by the running service and by retained revisions 5 and 6.***

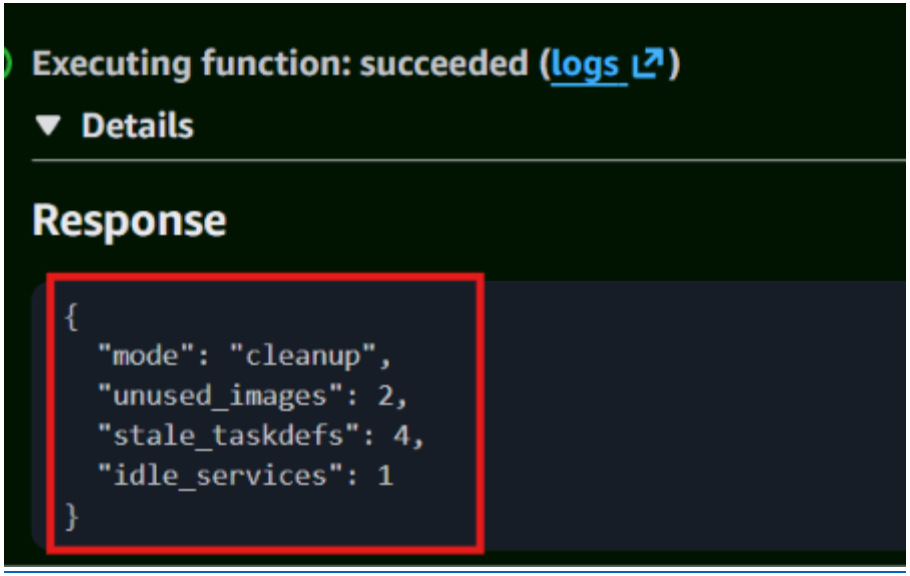
Image v4 ***not flagged: It is protected as the newest image (***KEEP_LATEST_IMAGES=1***).***

Task definition revisions 1 to 4 flagged: They fall outside the two newest revisions and are not in use.

guardian-demo-svc ***flagged as idle: The Nginx container receives no traffic and uses almost no CPU.***

Step VI: Run Cleanup Mode

- Review the dry-run report and confirm you are happy with what it lists.
- In the Lambda Test tab, create a second test event:
  - Event name: cleanup
  - Event JSON: {"mode": "cleanup"}
- Click Test.



- Verify the email now says DELETED instead of FOUND.

AWS Notifications <no-reply@sns.amazonaws.com>
to me

ECS Guardian report (cleanup mode)

1) Unused ECR images DELETED: 2 (0.05 GB, about $0.00/month)
- guardian-demo-app v3 (0 days old)
- guardian-demo-app v1 (0 days old)

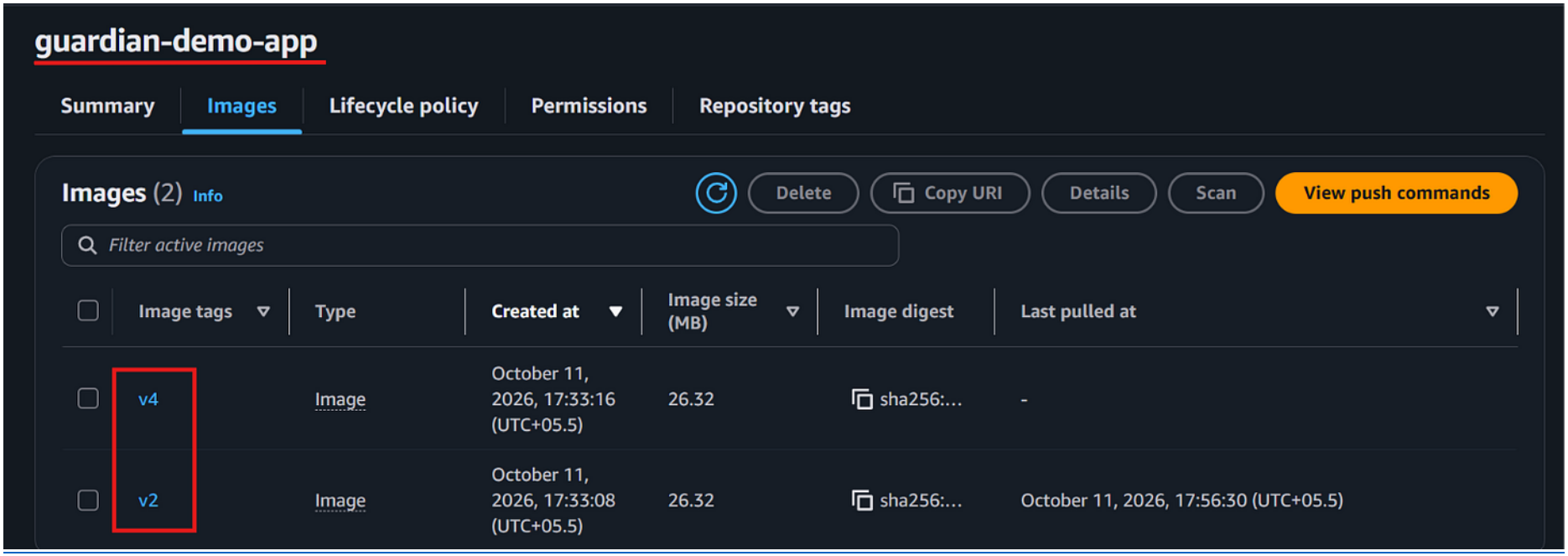
2) Stale task definition revisions DELETED: 4
...

- guardian-demo-task:4
- guardian-demo-task:3
- guardian-demo-task:2
- guardian-demo-task:1

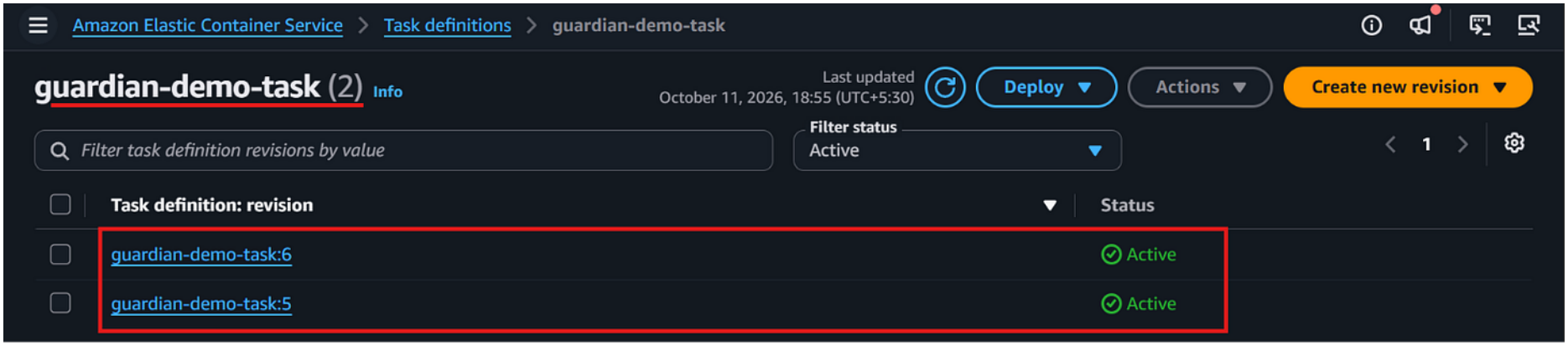
3) Idle ECS services (max CPU under 5.0% for 1 days): 1
- guardian-demo-cluster/guardian-demo-svc: 1 task(s), max CPU 0.7%, costs about $9.01/month

Possible savings: ECR about $0.00/month, idle services up to about $6.33/month if stopped nights and weekends.
Idle services are never touched automatically. Review them yourself.

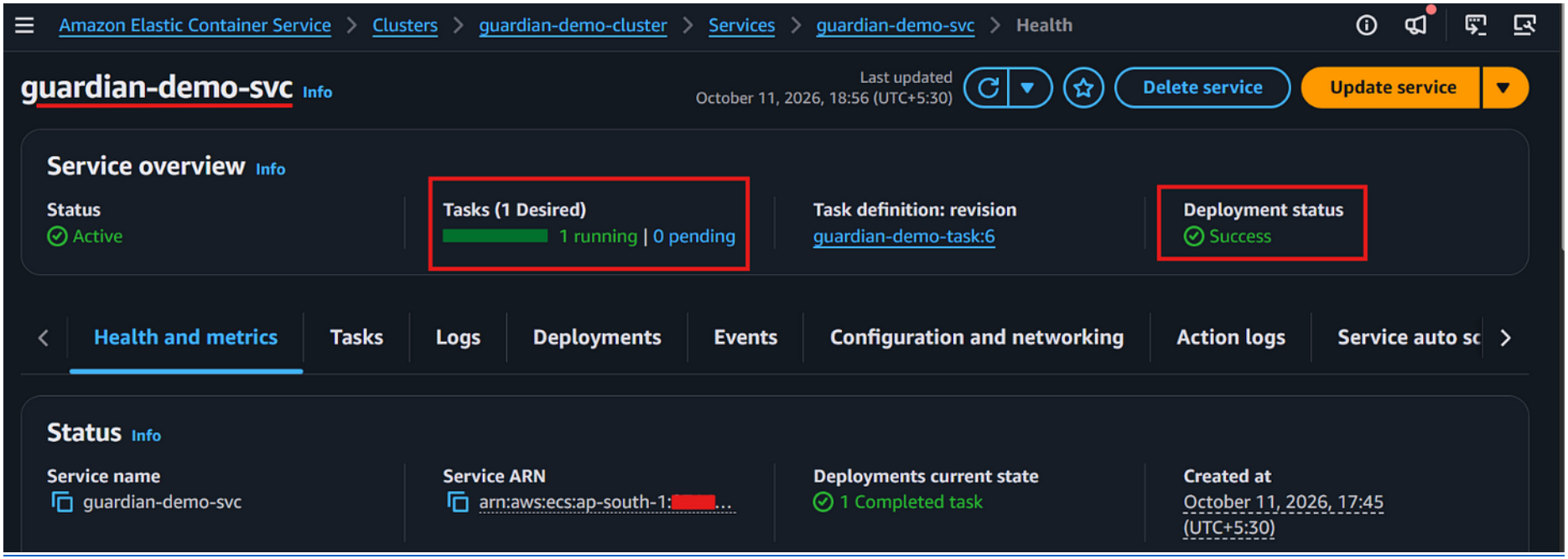
- Navigate to ECR → guardian-demo-app and verify only v2 and v4 remain.



- Navigate to ECS → Task definitions → guardian-demo-task and verify revisions 1 to 4 show Inactive, while 5 and 6 are still Active.



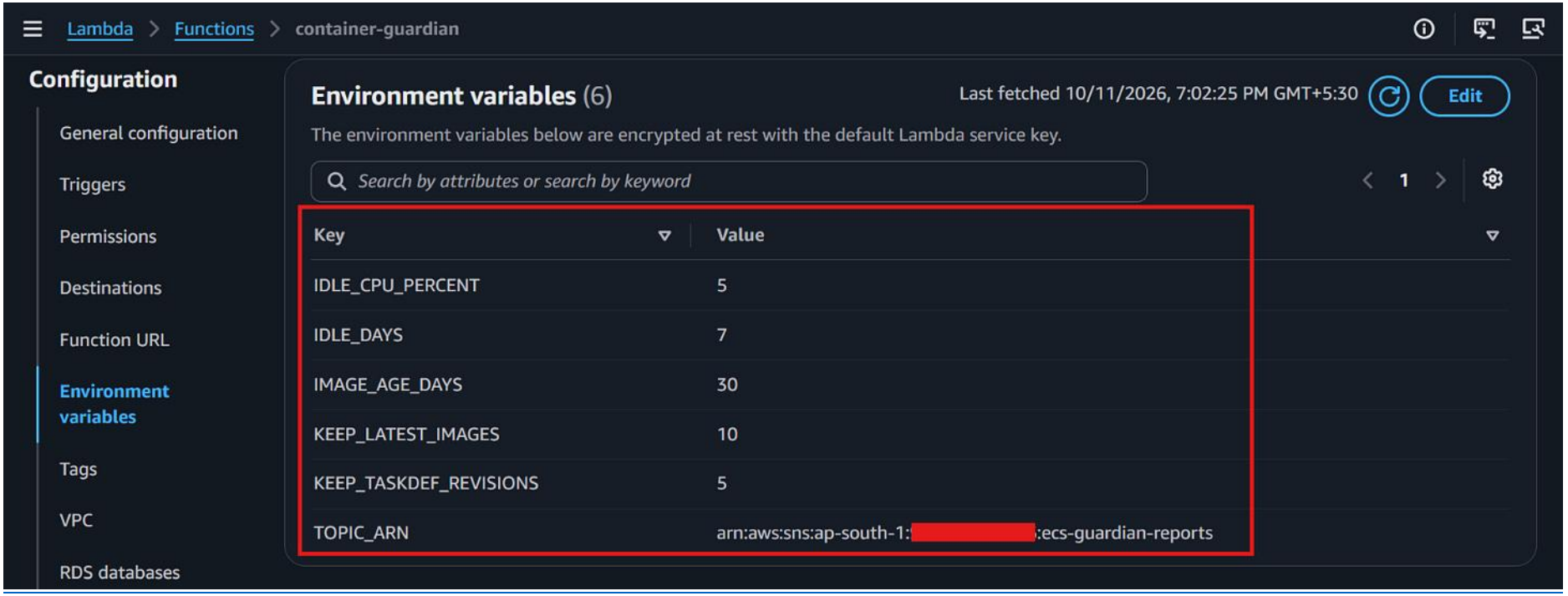
- Open ECS → guardian-demo-svc and verify the service is still running normally.



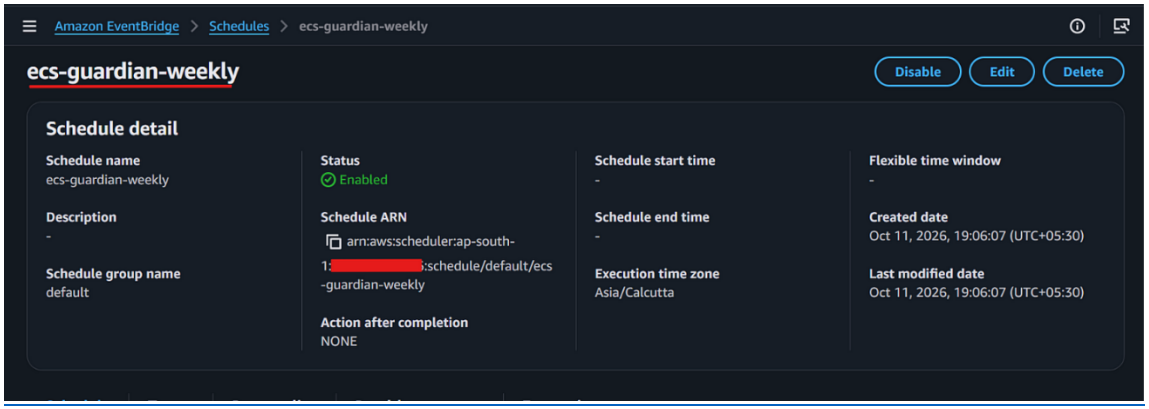
Cleanup mode recalculates everything at the moment you run it. It deletes what is unused right now, not what an older report listed.

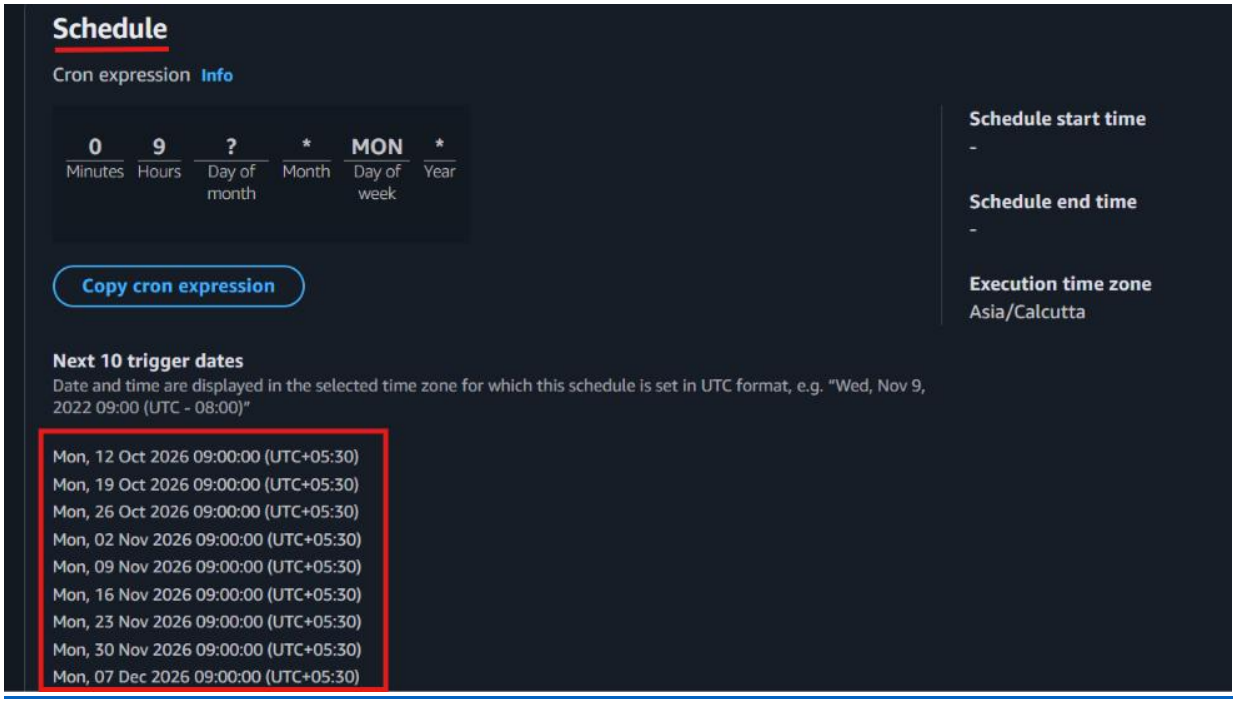
Step VII: Schedule Weekly Reports

- In Lambda → Configuration → Environment variables, change to production values:
  - IDLE_DAYS: 7
  - IMAGE_AGE_DAYS: 30
  - KEEP_LATEST_IMAGES: 10
  - KEEP_TASKDEF_REVISIONS: 5
- Click Save.



- Navigate to Amazon EventBridge → Scheduler → Schedules → Create schedule.
- Configure the following:
  - Schedule name: ecs-guardian-weekly
  - Occurrence: Recurring schedule
  - Schedule type: Cron-based schedule
  - Cron expression: 0 9 ? * MON * (every Monday, 9:00 AM)
  - Time zone: Asia/Kolkata
  - Flexible time window: Off
- Click Next. For the target, select Templated targets → AWS Lambda:
  - Function: container-guardian
  - Payload: {"mode": "report"}
- Let EventBridge create a new execution role for the schedule.
- Click Create schedule.
- Verify the schedule status shows Enabled.

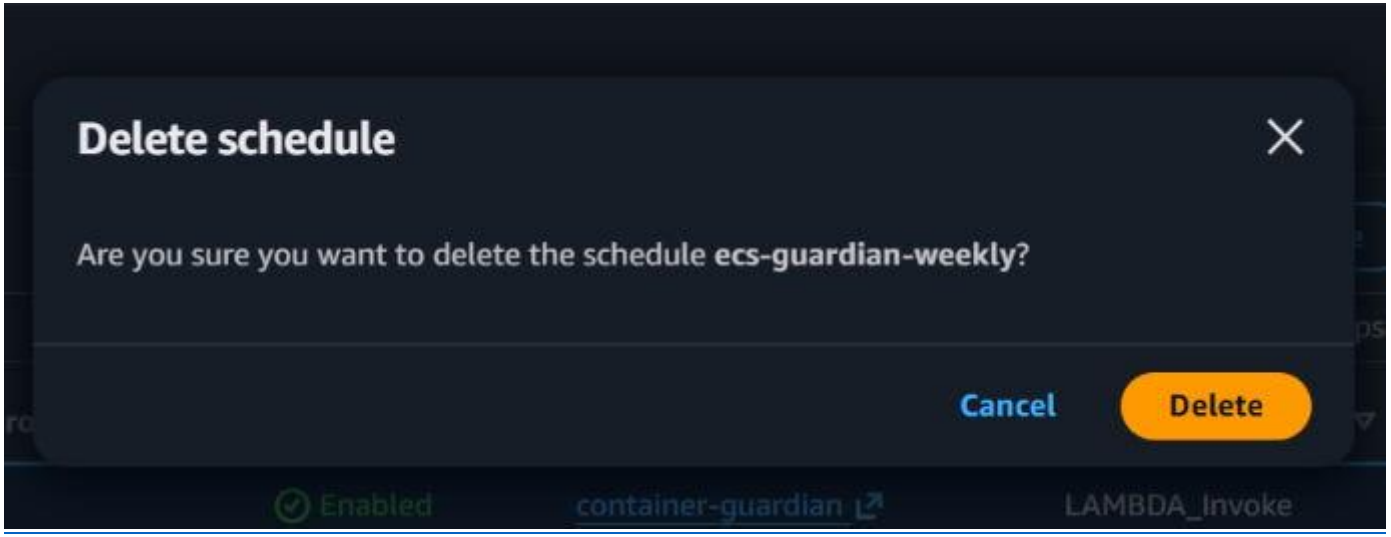

Amazon EventBridge > Schedules > ecs-guardian-weekly
ecs-guardian-weekly
Disable
Edit
Delete
Schedule detail
Schedule name
ecs-guardian-weekly
Description
-
Schedule group name
default
Status
Enabled
Schedule ARN
arn:aws:scheduler:ap-south-
1:[redacted]:schedule/default/ecs
-guardian-weekly
Action after completion
NONE
Schedule start time
-
Schedule end time
-
Execution time zone
Asia/Calcutta
Flexible time window
-
Created date
Oct 11, 2026, 19:06:07 (UTC+05:30)
Last modified date
Oct 11, 2026, 19:06:07 (UTC+05:30)


Schedule
Cron expression Info
0 Minutes
9 Hours
? Day of month
* Month
MON Day of week
* Year
Copy cron expression
Schedule start time
-
Schedule end time
-
Execution time zone
Asia/Calcutta
Next 10 trigger dates
Date and time are displayed in the selected time zone for which this schedule is set in UTC format, e.g. "Wed, Nov 9, 2022 09:00 (UTC - 08:00)"
Mon, 12 Oct 2026 09:00:00 (UTC+05:30)
Mon, 19 Oct 2026 09:00:00 (UTC+05:30)
Mon, 26 Oct 2026 09:00:00 (UTC+05:30)
Mon, 02 Nov 2026 09:00:00 (UTC+05:30)
Mon, 09 Nov 2026 09:00:00 (UTC+05:30)
Mon, 16 Nov 2026 09:00:00 (UTC+05:30)
Mon, 23 Nov 2026 09:00:00 (UTC+05:30)
Mon, 30 Nov 2026 09:00:00 (UTC+05:30)
Mon, 07 Dec 2026 09:00:00 (UTC+05:30)

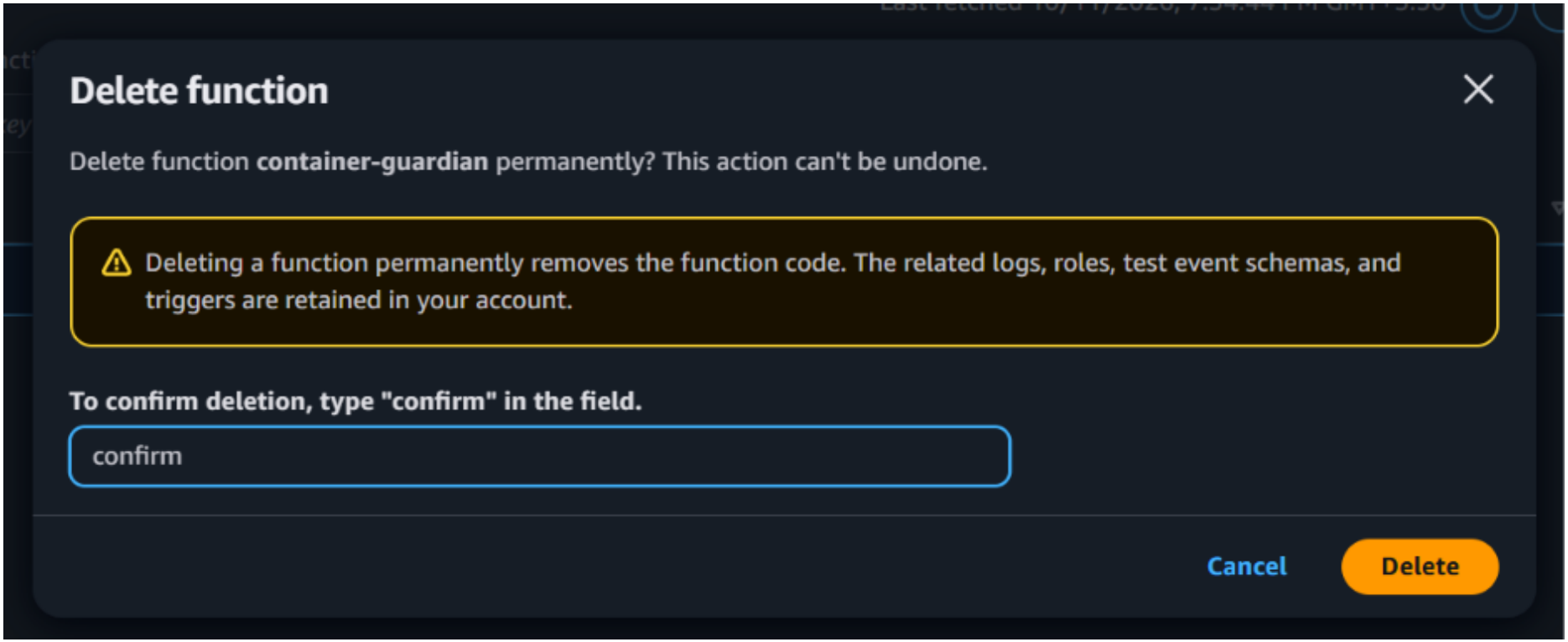
Step VIII: Clean Up Resources

Delete the following resources in this order:

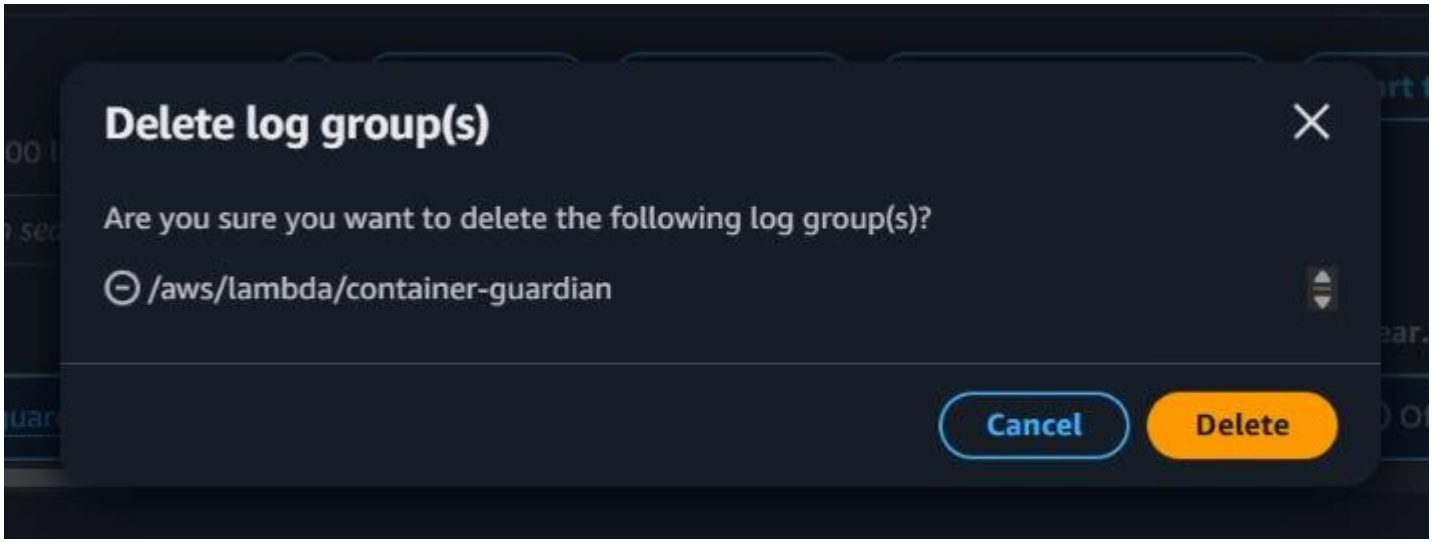
- Delete the EventBridge schedule ecs-guardian-weekly.



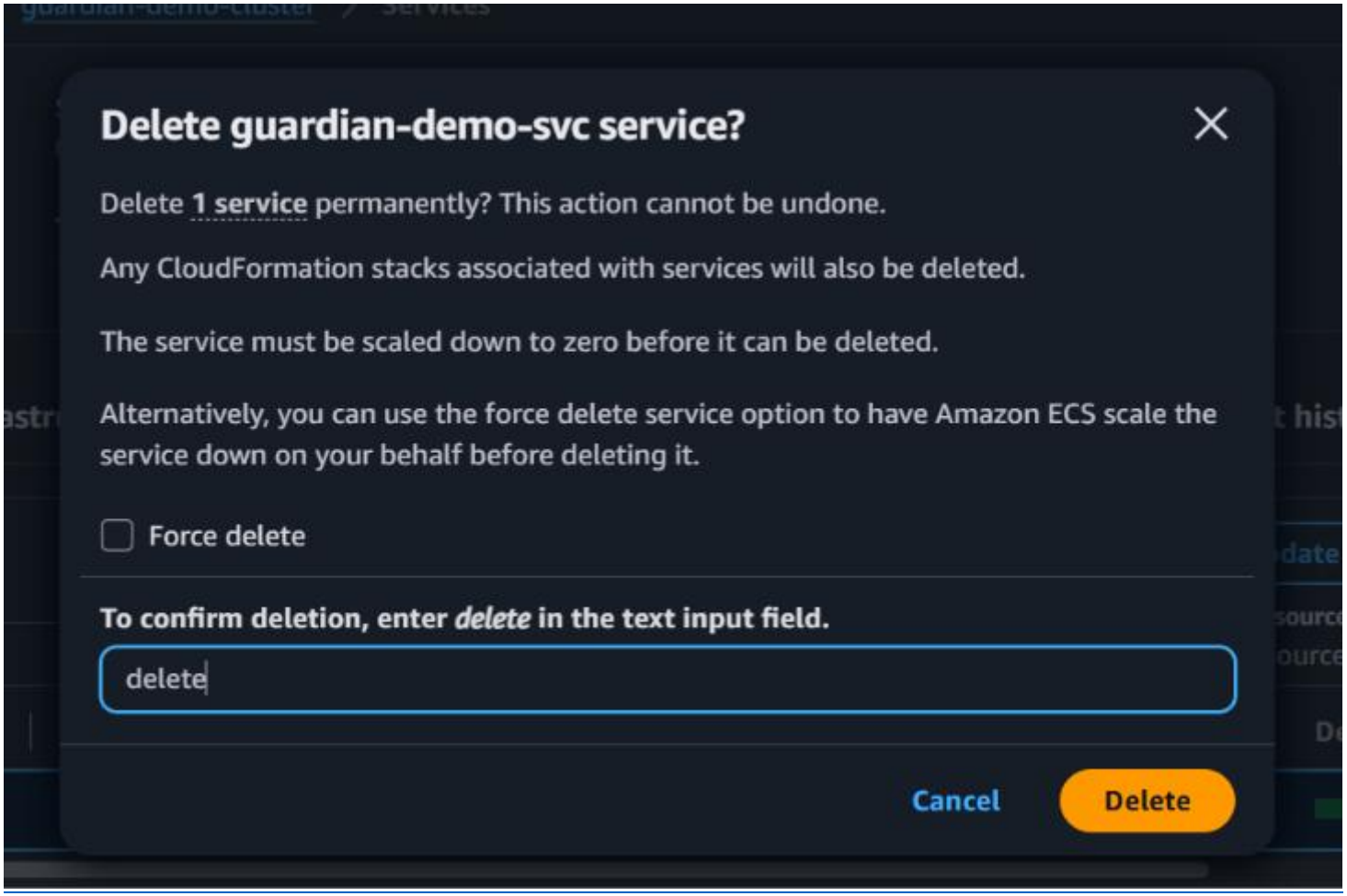
- Delete the Lambda function container-guardian.



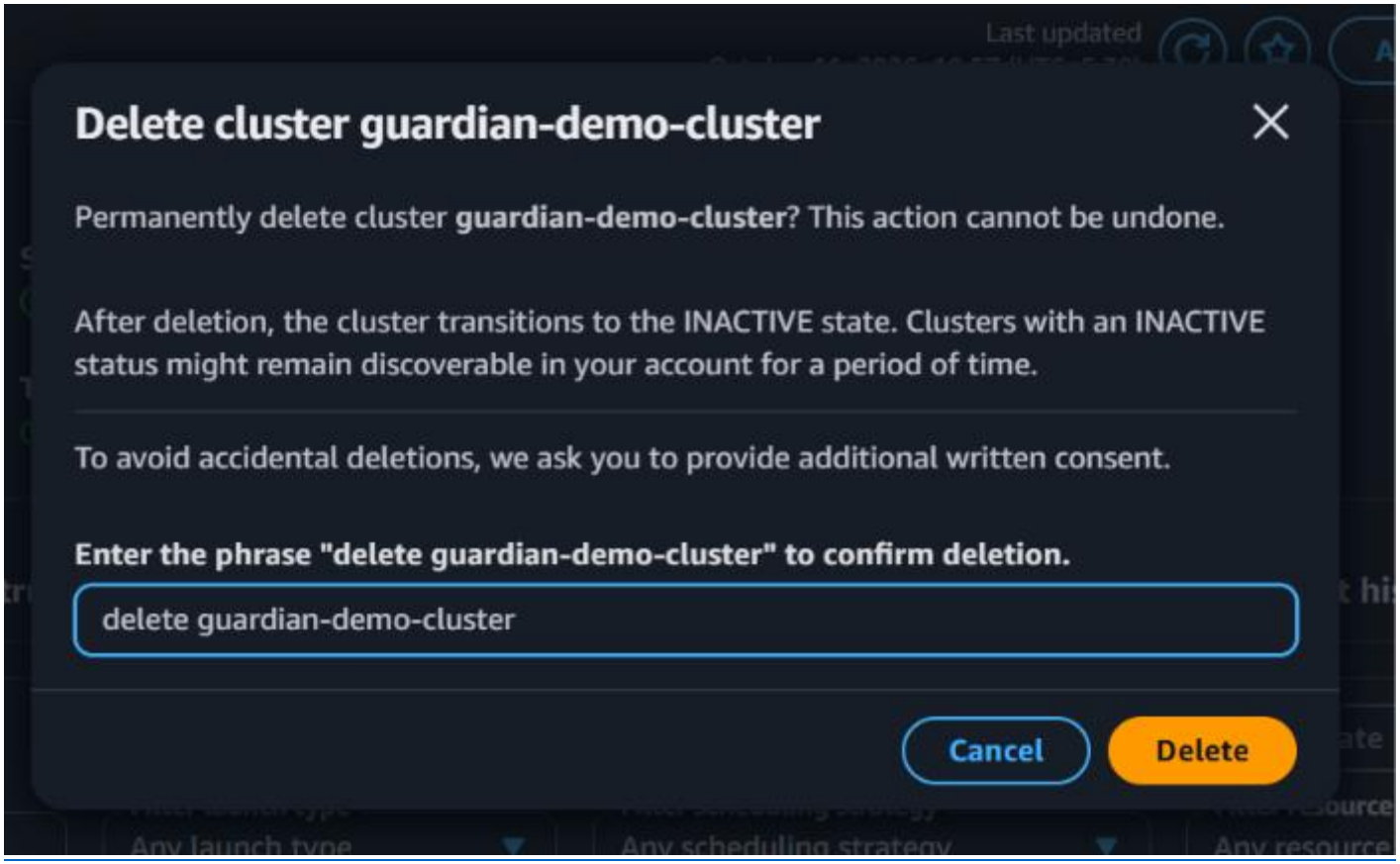
- Delete the CloudWatch log group /aws/lambda/container-guardian.



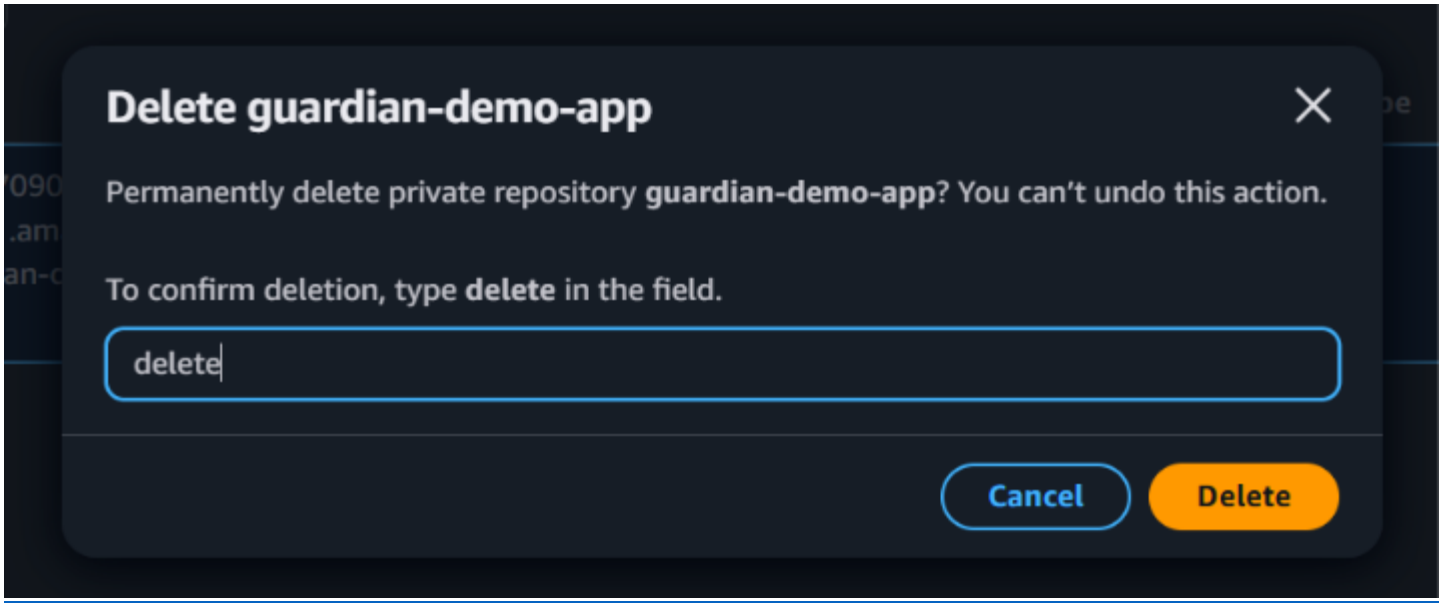
- Delete the ECS service guardian-demo-svc (select Force delete if asked), then verify the task has stopped.



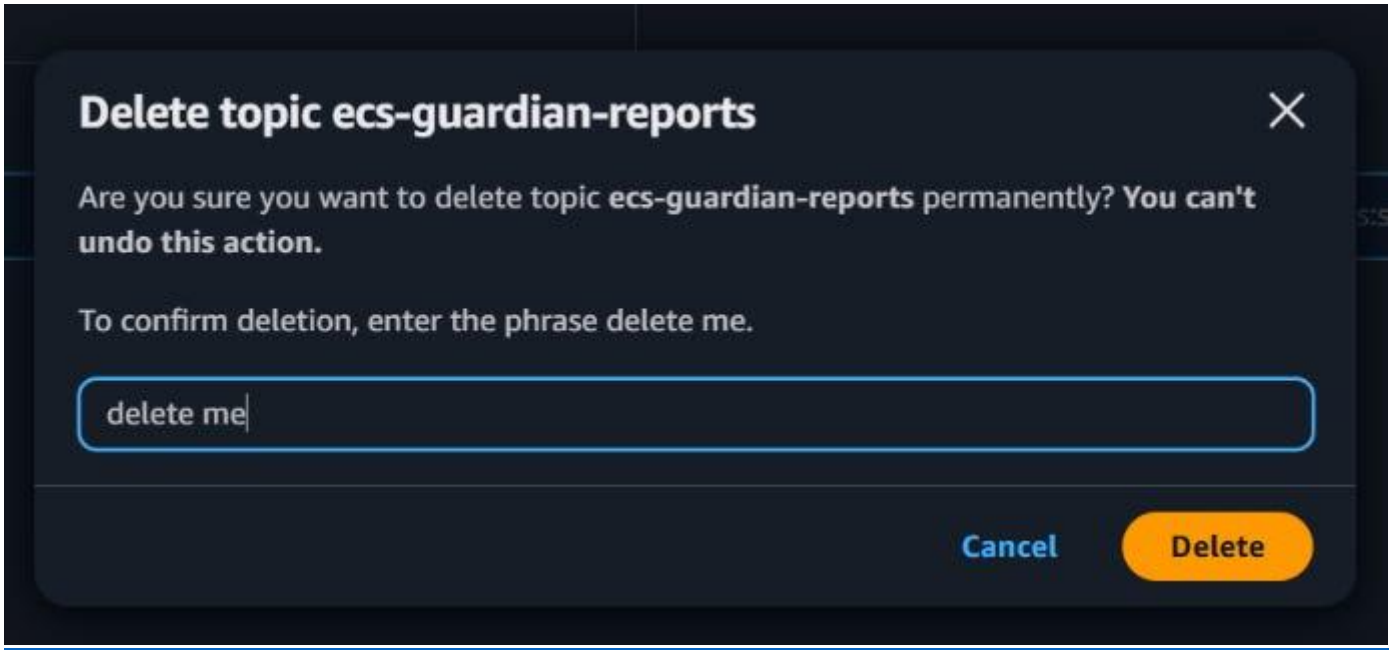
- Delete the ECS cluster guardian-demo-cluster.



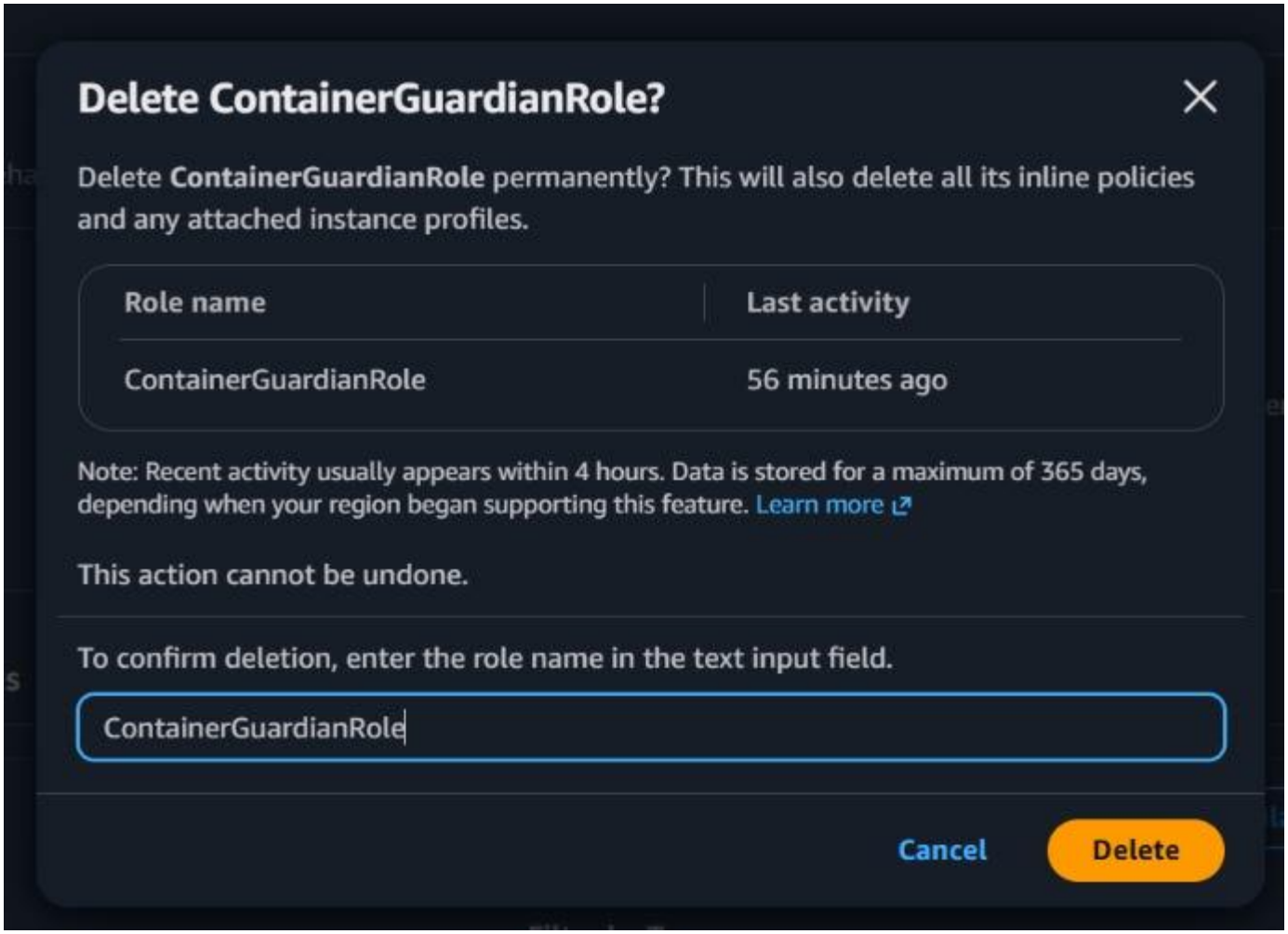
- Delete the ECR repository guardian-demo-app (this also deletes its remaining images).



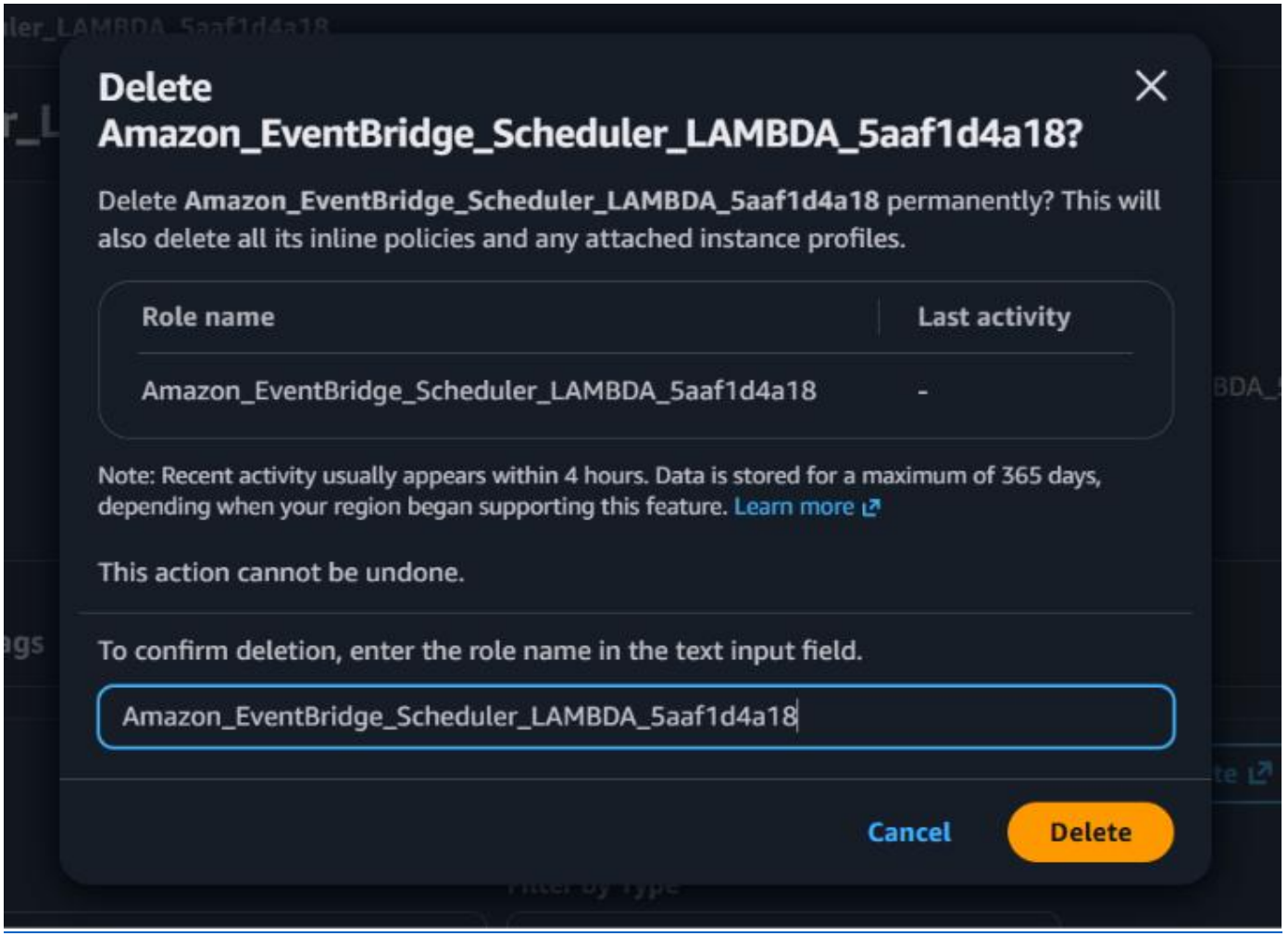
- Delete the SNS topic ecs-guardian-reports and its subscription.



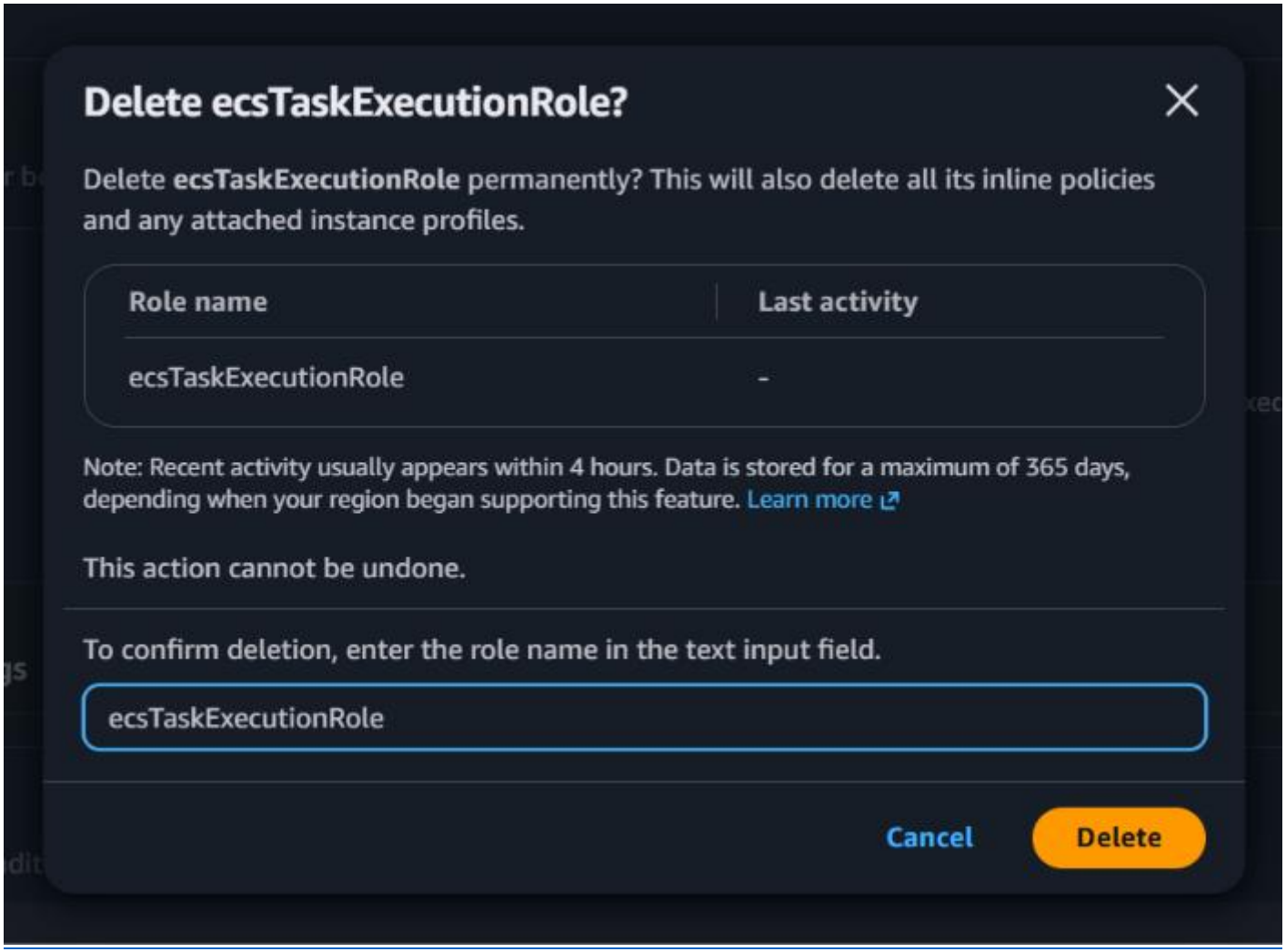
- Delete the IAM role ContainerGuardianRole



- Delete the auto-created EventBridge Scheduler execution role (named like Amazon_EventBridge_Scheduler_LAMBDA_...).



- Delete ecsTaskExecutionRole



🔐 Security and Least-Privilege Considerations

ECS Guardian can delete resources, so its permissions are scoped carefully.

How the IAM policy is structured

- **Read access is broad, write access is narrow.** Most actions are Describe* and List* calls. The only destructive actions are ecr:BatchDeleteImage and ecs:DeregisterTaskDefinition, and both run only in cleanup mode.
- **Notifications are limited to one topic.** sns:Publish is scoped to the ARN of ecs-guardian-reports, so the function cannot publish to any other topic.
- **No access to workloads.** The role cannot stop tasks, update services, or modify clusters. Idle services are reported, never changed.
- **Separate execution roles.** The Lambda role (ContainerGuardianRole) is distinct from ecsTaskExecutionRole, which only the Fargate tasks use.

🛡️ Safety Guardrails

The Guardian is conservative by design. Several independent layers must all agree before anything is removed.

1. Report before delete
The default mode is report, a dry run that deletes nothing. Cleanup runs only when a person invokes it with {"mode": "cleanup"}, and it recalculates everything at that moment rather than replaying an old report.

2. Keep-latest-N rules

- KEEP_LATEST_IMAGES always protects the newest N images in each repository.
- KEEP_TASKDEF_REVISIONS always protects the newest N revisions in each task definition family.

3. Age threshold
An image must be at least IMAGE_AGE_DAYS old before it can be flagged, so recent pushes are never touched.

4. Usage protection
An image is never flagged if it is referenced by:

- a running task,
- a task definition used by a service, or
- a task definition revision that is being kept.

In the demo, this is why v2 survives (it is running and used by revisions 5 and 6) and v4 survives (it is the newest).

5. The guardian:ignore=true **tag**
Add this tag to any ECR repository or ECS service to exempt it from scanning and cleanup.

✅ Best Practices and Lessons Learned

- **Always start with a dry run.** Read the report carefully before triggering cleanup.
- **Test with aggressive thresholds, then switch to conservative ones.** Low values like IMAGE_AGE_DAYS=0 make the demo resources show up immediately, but production should use values like 30 days, 10 images, and 5 revisions.
- **Tag protected resources early.** Apply guardian:ignore=true to anything that must never be cleaned, such as rollback images or shared base images.
- **Wait for CloudWatch data.** Idle detection needs metrics, so allow about 10 minutes after a service starts, and use IDLE_DAYS=7 in production to avoid false positives.
- **Use correct pricing.** The Fargate price variables default to US East rates, so set VCPU_HOUR and GB_HOUR for your region to get accurate savings estimates.
- **Keep the schedule on report mode.** Automate reports, but keep cleanup as a manual, deliberate action.
- **Remember that unique images are what count.** Multiple tags can point to the same image digest, and ECR storage is billed per image, so deleting a tag alone may not reduce cost.
- **Clean up lab resources promptly.** A running Fargate task keeps billing until the service is deleted.

💻 Source Code & Repository

For the complete ECS Guardian project, including the Python Lambda function, IAM policies, shell scripts, architecture documentation, deployment and cleanup guides, security practices, and execution outcomes, please refer to the accompanying GitHub repository.

🔗 **GitHub Repository: ecs-guardian**

📌 Conclusion

Container platforms accumulate waste quietly. Every deployment leaves behind an image and a task definition revision, and forgotten services keep running around the clock. ECS Guardian addresses this with a single serverless Lambda function that scans Amazon ECR, Amazon ECS, and Amazon CloudWatch, identifies what is unused, and emails a cost-savings report through Amazon SNS every week.

The key design principle is to be conservative and keep a human in the loop. The Guardian reports by default, protects anything in use or recently created, exempts tagged resources, and never touches idle services automatically. Deletion happens only when someone reviews the findings and explicitly triggers cleanup mode.

The result is a low-cost, low-risk way to keep a container platform tidy and reduce storage and compute spend, with no servers to manage.

🔑 Key Takeaways

- **Waste builds up silently.** Old images, stale task definition revisions, and idle services all add to your AWS bill.
- **One Lambda function covers it all.** It scans ECR, ECS, and CloudWatch, then reports through SNS on a weekly EventBridge schedule.
- **Safety comes first.** Dry run by default, keep-latest-N rules, usage checks, and the guardian:ignore=true tag prevent accidental deletion.
- **Humans make the final call.** Cleanup is manual, and idle services are only ever reported.

🤝 Let's Connect

Open to ideas, feedback, and meaningful collaboration:

LinkedIn: https://www.linkedin.com/in/khushi-nandwani/

🌐 Explore More

To see more of my projects and technical work, feel free to visit:

GitHub: https://github.com/Knandwani07

Dev Community: https://dev.to/khushi_nandwani07

Medium: https://medium.com/@khushinandwanii